BidMate RAG 기술 보고서

— 공공입찰 **RFP** 특화 초정밀 질의응답 시스템

목차

1. 프로젝트 개요

1.1 연구 목적 및 범위
1.2 핵심 성공 지표 (KPI) 설정
1.3 팀 구성 및 R&R

2. 개발 환경 및 베이스라인

2.1 인프라 아키텍처 (GCP L4 GPU 구조)
2.2 기술 스택 및 라이브러리 표준화
2.3 베이스라인 (Naïve RAG) 파이프라인 수립

3. 데이터 엔지니어링 및 구조화

3.1 RFP 데이터 무결성 확보를 위한 메타데이터 보정 전략
3.2 B2G 문서 구조 보존형 전처리 파이프라인
3.3 문자열 분할과 의미 기반 동적 청킹 스위칭 메커니즘
3.4 메타데이터 워터마크 주입을 통한 도메인 컨텍스트 확립

4. RAG 시스템 아키텍처 (Scenario Design)

4.1 시나리오 A: 로컬 거대 언어 모델 최적화 — Native FP16 모델 원천 지능 보존 전략
4.2 시나리오 B: 고성능 상용 API 파이프라인 — 품질 천장 확인용 레퍼런스 아키텍처
4.3 초정밀 벡터 데이터베이스(Vector DB) 메타 필터 레이어 구축

5. 검색 및 생성 고도화 전략

5.1 한국어 도메인 특화 임베딩(BAAI/bge-m3) 모델 성능 평가
5.2 의도 추론 기반 적응형 하이브리드 검색 오케스트레이션 (Retriever)

5.3 5-Tier 구조 기반 초정밀 프롬프트 엔지니어링 및 페르소나 주입
5.4 구조적 파이프라인 개편: Cross-Encoder의 전략적 배제 및 하이브리드 + Soft Boost 최적화

6. 성능 평가 및 비교 분석

6.1 시나리오 B (상용 API) 정량 성능 종합 분석
6.2 시나리오 A (로컬 LLM) 정량 성능 종합 분석
6.4 시나리오 A 프롬프트 최적화 케이스 분석

7. 기술 의사결정 및 트러블슈팅

7.1 검색 파이프라인 고도화
  7.1.1 문제 인식: Dense 단일 검색의 한계
  7.1.2 Cross-Encoder (CE) 전략적 미적용 (배제) 결론
  7.1.3 하이브리드 검색 전환: Dense + Sparse + RRF 융합
  7.1.4 Hard Filter에서 Soft Boost로의 전환
7.2 평가 체계 수립: Judge v1에서 v2로의 전환
  7.2.1 문제: MRR = 0.000이 의미하는 것
  7.2.2 Judge v2 재설계
7.3 멀티턴 대화 엔진 구축
  7.3.1 아키텍처 분리: 단일턴과 멀티턴의 이원화
  7.3.2 LLM Rewrite 엔진의 구현과 안정화
7.4 프롬프트 엔지니어링 체계화
  7.4.1 단일 프롬프트에서 5-Tier 조합형 아키텍처로의 전환
  7.4.2 Citation 사전 주입 시스템
7.5 데이터 품질 및 인덱싱 안정화
  7.5.1 Document ID 매칭 오류 해결
7.6 비용 추적 및 최적화 (Cost Governance)
7.7 고도화 전후 종합 비교

8. 종합 평가 및 제언

8.1 프로젝트 성과 종합
8.2 시나리오 A/B 모델별 실험 특성 및 한계돌파 인사이트
8.3 현재 한계 및 향후 고도화 로드맵
8.4 종합 결론 및 시사점

1. 프로젝트 개요

1.1 연구 목적 및 범위

연구의 출발점: 공공입찰 컨설턴트의 고질적 비효율

대한민국 공공입찰 시장에서 제안요청서(RFP, Request for Proposal)는 사업 수주의 핵심 문서다. 컨설턴트는 수백 페이지에 달하는 RFP 문서를 읽고, 핵심 요구사항을 추출하며, 평가 기준과 자격 조건을 파악해야 한다. 그러나 이 과정은 전적으로 수작업에 의존하고 있었다.

BidMate RAG 프로젝트는 이러한 비효율을 근본적으로 해결하기 위해 탄생하였다. 공공입찰 **RFP 100**건을 대상으로 한 **RAG(Retrieval-Augmented Generation)** 질의응답 시스템을 구축하여, 입찰메이트 컨설팅 사내 도구로 활용하는 것이 본 프로젝트의 최종 목표다.

프로젝트 범위 정의

본 프로젝트는 단순한 검색 엔진을 넘어, "질문에 대해 문서 근거를 명시하며 정확하게 답변하는 지식 근거주의**(Evidence-grounded)** 체계"를 구현한다. 그 기술적 범위는 다음과 같다.

범위 구분	세부 내용
데이터 소스	공공입찰 RFP 원본 문서 101건 (HWP 96건, PDF 4건, DOCX 1건)
핵심 파이프라인	파싱 → 정제 → 청킹 → 인덱싱 → 검색 → 생성 → 평가 → 리포트
운영 시나리오	시나리오 A (로컬 오픈소스 LLM) / 시나리오 B (상용 API LLM)
최종 산출물	CLI 기반 평가 도구 + Streamlit 디버깅 UI + Next.js 사용자 채팅 인터페이스

자동화 수준	YAML 1장 정의 → CLI 1줄 실행 → 비용·메트릭 자동 기록 → 노션 친화 마크다운 리포트 자동 생성

프로젝트가 해결하는 비즈니스 과제

본 시스템을 통해 컨설턴트는 다음과 같은 12종의 슬래시 커맨드를 활용하여 즉각적으로 RFP 핵심 정보에 접근할 수 있다.

- `/요약` — 문서 전체 요약
- `/요구사항` — 기술 요구사항 추출
- `/일정` — 사업 일정 파악
- `/예산` — 예산 정보 조회
- `/비교` — 복수 문서 간 비교 분석
- `/자격요건` — 참여 자격 조건 확인
- `/평가기준` — 평가 배점 기준 추출
- `/리스크` — 리스크 요소 식별
- `/기본정보` — 사업 기본 정보 조회
- `/제출서류` — 제출 서류 목록 확인
- `/도움말` — 사용법 안내
- `/초기화` — 세션 초기화

이처럼 본 프로젝트는 단순한 기술 시연이 아니라, 실무 컨설턴트의 업무 흐름에 직접 통합되는 프로덕션 수준의 **RAG** 도구를 지향한다.

1.2 핵심 성공 지표 (KPI) 설정

파이프라인 단계별 평가 체계

BidMate RAG 시스템의 품질은 단일 지표로 측정할 수 없다. 파이프라인의 각 단계—검색(Retrieval), 재정렬(Reranking), 생성(Generation)—마다 독립된 평가 축이 존재하며, 이를 종합적으로 관리하기 위해 다층 KPI 체계를 설계하였다.

검색 성능 지표 **(Retrieval Metrics)**

지표	정의	목표 수준
Hit Rate@5	상위 5개 검색 결과 중 정답 문서가 1개 이상 포함된 비율	≥ 0.85
MRR (Mean Reciprocal Rank)	정답 문서가 검색 결과에서 최초로 등장하는 순위의 역수 평균	≥ 0.70
nDCG@5	검색 결과의 정답 문서 순위 분포가 이상적 순위 분포에 얼마나 근접하는가	≥ 0.65

생성 품질 지표 **(Generation Metrics — RAGAS** 기반**)**

본 프로젝트는 전통적인 BLEU/ROUGE 스코어가 생성형 AI 답변 평가에 부적합하다는 판단 하에, RAGAS 프레임워크에 기반한 **LLM-as-a-Judge** 4대 지표를 채택하였다.

지표	평가 대상	채점 척도
Faithfulness (충실도**)**	답변의 모든 주장이 검색된 컨텍스트에만 근거하는가?	0.0 / 0.25 / 0.5 / 0.75 / 1.0
Answer Relevance (답변 적합성**)**	답변이 질문에 직접적으로 답하는가?	0.0 / 0.25 / 0.5 / 0.75 / 1.0
Context Precision (문맥 정밀도**)**	검색된 컨텍스트 중 답변에 실제 사용된 비율	0.0 / 0.25 / 0.5 / 0.75 / 1.0
Context Recall (문맥 재현율**)**	정답에 필요한 정보가 검색된 컨텍스트에 포함된 비율	0.0 / 0.25 / 0.5 / 0.75 / 1.0

운영 효율성 지표 **(Operational Metrics)**

지표	측정 대상	비즈니스 임계값
Latency(A/B)	질문 입력부터 답변 생성 완료까지의 소요 시간	≤ 90/30초
Cost per Query	1건의 질의응답에 소모되는 API 비용 (USD)	≤ $0.05
Token Efficiency	입력 토큰 대비 유의미한 답변 생성 비율	모니터링

모든 지표는 실험 실행 시 자동으로 JSONL 로그에 기록되며, `{run_id}.meta.json` 파일에 Git 커밋 해시, 설정 스냅샷과 함께 영구 보존된다. 이를 통해 어떤 설정 변경이 어떤 지표 변화를 야기했는지 완전한 추적성(Traceability)을 확보하였다.

1.3 팀 구성 및 R&R

협업 거버넌스 구조

BidMate RAG 프로젝트는 체계적인 Git 브랜치 전략과 역할 분담을 통해 병렬 개발을 수행하였다.

브랜치 전략

본 프로젝트는 `main → develop → feature branch` 3단계 브랜치 전략을 채택하였다.

브랜치 유형	용도	네이밍 규칙
`main`	배포 및 제출 기준선. 직접 작업 금지	—
`develop`	팀 공통 개발 기준선. 기능 브랜치 분기점	—
`feat/<initial>/<topic>`	개인 기능 개발	`feat/jh/hwp-loader`
`fix/<initial>/<topic>`	버그 수정	`fix/mk/metadata-filter`
`experiment/<initial>/<topic>`	실험용 분기	`experiment/ay/chunking-test`

역할 분담 체계

팀은 다음과 같은 R&R(Role & Responsibility) 구조로 운영되었다.

이름	역할	주요 담당 영역
김도민	**Lead Role:** Project Manager	• 프로젝트 일정 관리와 역할 분배, 우선순위 조정 • 회의 주도, 노션/이슈보드 운영, 협업 체계 관리

	Sub Role: Evaluation / Documentation	• 마일스톤 및 제출 일정 관리, A/B 시나리오 진행 상황 통합, 초기 파이프라인 구축
김예주	**Lead Role:** Retrieval / Embedding **Sub Role:** Chunking / Metadata	• 임베딩 모델과 Vector DB 선정 • 문서 임베딩 생성과 similarity search, Top-k, MMR, re-ranking 기반 검색 성능 고도화 • 검색 성능 비교 실험과 retrieval 파이프라인 개선
김정우	**Lead Role:** Chunking / Metadata **Sub Role:** Document / Data	• 청킹 전략 설계와 chunk size / overlap 실험 • 섹션·문단 기반 분할 검토와 문서 구조 기반 chunking 개선 • 메타데이터 스키마 설계, 기관명·사업명 정규화, 메타데이터 필터링 기준 정리
김준혁	**Lead Role:** Document / Data **Sub Role:** Generation / Prompt	• PDF/HWP 문서 로딩, 텍스트 추출 및 정제 • data_list.csv 메타데이터 확인 및 매핑, 문서 구조 파악 • 문서 전처리 흐름 정리와 입력 데이터 정합성 관리
손승만	**Lead Role:** Generation / Prompt **Sub Role:** Evaluation / Documentation	• LLM 선정과 프롬프트 설계, 답변 포맷·톤·길이 제어 • 검색된 컨텍스트 기반 응답 생성과 후속 질문 대응 • hallucination 억제와 응답 품질 개선 • 평가셋 80문항 제작 및 평가셋 배치 전략 기획

모든 개발 작업은 **Git Worktree** 기반으로 독립된 작업 디렉토리에서 수행되었으며, 기능 완성 후 `develop` 브랜치로 PR(Pull Request)을 통해 병합하는 절차를 준수하였다.

---

2. 개발 환경 및 베이스라인

2.1 인프라 아키텍처 (GCP L4 GPU 구조)

컴퓨팅 환경 설계 철학

본 프로젝트는 두 가지 극단적인 운영 환경을 동시에 지원해야 하는 도전에 직면했다. 시나리오 A에서는 외부 API 접근 없이 로컬 GPU에서 거대 언어 모델을 구동해야 하고, 시나리오 B에서는 OpenAI API를 통해 클라우드 기반의 고성능 추론을 수행해야 한다. 이 두 시나리오를 하나의 통합된 파이프라인 위에서 구동하기 위해, 다음과 같은 인프라를 구축하였다.

하드웨어 스펙

구성 요소	사양
클라우드 플랫폼	Google Cloud Platform (GCP)
GPU	NVIDIA L4 Tensor Core GPU
VRAM	24GB GDDR6
CPU	vCPU (GCP Compute Engine 기반)
스토리지	SSD Persistent Disk
컨테이너	Docker 기반 격리 환경

메모리 관리 전략

로컬 LLM (gemma, Qwen2.5-7B 등) 구동 시 24GB VRAM은 결코 넉넉하지 않다. 특히 7B 파라미터 모델을 Full Precision (FP32)으로 로드하면 약 28GB가 필요하여 단일 GPU에서 구동이 불가능하다. 이를 극복하기 위해 다음과 같은 최적화 기법을 적용하였다.

- **4-bit Quantization (bitsandbytes)**: 모델 가중치를 4비트로 양자화하여 VRAM 사용량을 약 1/4로 축소
- **vLLM/Ollama** 기반 추론 서버: 모델 로딩과 추론을 분리하여 메모리 활용 효율 극대화
- **Batch Inference** 제어: 동시 처리 크기를 VRAM 한계 내에서 동적 조절

2.2 기술 스택 및 라이브러리 표준화

코어 프레임워크 구성

본 프로젝트의 기술 스택은 재현 가능성**(Reproducibility)**과 모듈 교체 용이성을 최우선 원칙으로 선정하였다. Python 3.12 이상을 기준으로, 핵심 의존성은 `pyproject.toml`에서 단일 소스로 관리된다.

핵심 라이브러리 일람표

영역	라이브러리	버전	역할
벡터 **DB**	ChromaDB	≥ 1.5.6	벡터 저장·검색, 메타데이터 필터링
텍스트 분할	langchain-text-splitters	≥ 1.1.1	Recursive/Markdown 기반 청킹
시맨틱 청킹	langchain-experimental	≥ 0.4.1	SemanticChunker (임베딩 유사도 기반 분할)
임베딩	langchain-openai	≥ 1.1.12	OpenAI 임베딩 API 연동
LLM API	openai	≥ 2.30.0	GPT-5 시리즈 호출
데이터 처리	pandas	≥ 3.0.2	메타데이터·청크 DataFrame 관리
직렬화	pyarrow	≥ 21.0.0	Parquet 포맷 입출력
스키마 검증	pydantic	≥ 2.11.7	문서/청크/응답 데이터 모델
API 서버	FastAPI + uvicorn	≥ 0.115.0	사용자 UI 백엔드 API
토크나이저	tiktoken	≥ 0.12.0	토큰 수 계산 및 비용 추정
로컬 **ML**	bitsandbytes	≥ 0.49.2	4-bit 양자화 지원
파싱 **(HWP)**	hwp-hwpx-parser	≥ 1.0.0	HWP 문서 파싱 보조
시각화	matplotlib	≥ 3.10.8	실험 결과 시각화
UI (디버깅**)**	Streamlit	≥ 1.56.0	팀 디버깅용 3탭 인터페이스

의존성 그룹 분리

```
[dependency-groups]
├── dev        : pytest, ruff (lint/format)
```

```
├── notebook  : ipykernel, jupyter, matplotlib
├── ui        : streamlit
└── ml        : datasets, peft, sentence-transformers, transformers, trl
```

개발자는 `uv sync --group dev`로 기본 환경을, `uv sync --group ml`로 ML 학습 환경을 선택적으로 구성할 수 있으며, 이를 통해 불필요한 대형 패키지(transformers 등)의 설치를 회피하여 환경 세팅 시간을 단축하였다.

2.3 베이스라인 (Naïve RAG) 파이프라인 수립

깡통 RAG: 고도화 이전의 기준선

모든 고도화 전략의 효과를 정량적으로 입증하려면, 먼저 어떠한 최적화도 적용되지 않은 순수한 상태의 **RAG** 파이프라인이 어느 수준의 성능을 보이는지 측정해야 한다. 이것이 베이스라인(Baseline)이며, 본 시스템에서는 이를 **"Naïve RAG"**라 명명한다.

Naïve RAG 구성 요소

Naïve RAG는 다음과 같은 최소한의 구성으로 작동한다.

파이프라인 단계	**Naïve RAG** 설정	비고
파싱	Kordoc 기반 마크다운 변환	기본 전처리만 적용
청킹	Chunk Size 1,000자 / Overlap 150자	medium 프리셋
임베딩	BAAI/bge-m3	고정
검색	Dense 검색 (코사인 유사도) Top-5	Sparse 검색 미적용
재정렬	미적용	Cross-Encoder 없음
생성	기본 시스템 프롬프트	고도화된 페르소나·제약 조건 미삽입

멀티턴	미적용	대화 이력 상속 없음

Naïve RAG의 검색 흐름

Naïve RAG는 극도로 단순한 검색 로직을 따른다.

1. 사용자 질문이 입력된다
2. 질문에서 발주기관명이 감지되면 `where` 필터를 적용한다
3. 감지되지 않으면 필터 없이 전체 벡터 공간을 대상으로 검색한다
4. 코사인 유사도 기준 Top-5 청크를 반환한다
5. 반환된 컨텍스트와 질문을 LLM에 전달하여 답변을 생성한다

이 구조에서는 하이브리드 검색도, **Reranker**도, 멀티턴 대화 상속도, 팬아웃 검색도 존재하지 않는다. 오직 임베딩 벡터의 코사인 거리만으로 문서를 찾아오는, 말 그대로 '깡통' 상태다.

베이스라인의 관찰된 한계

이 Naïve RAG 상태에서 관찰된 핵심 한계점은 다음과 같다.

한계 유형	구체적 현상	근본 원인
기관 혼동	"A기관 사업 예산"을 물었는데 B기관 데이터를 인용	메타데이터 필터링의 부재
고유명사 실패	특정 모델 번호, 법률 조항 등을 벡터 유사도로 찾지 못함	Sparse (키워드) 검색 미적용
대명사 붕괴	"그 사업의 예산은?"이라는 후속 질문에 전혀 다른 사업 검색	멀티턴 맥락 상속 미적용
노이즈 상위 랭킹	질문과 무관한 청크가 상위권에 포진	Reranker 미적용
환각 발생	컨텍스트에 없는 내용을 LLM이 자체 생성	프롬프트 Constraints 미적용

이 한계점들은 이후 3장~5장에 걸쳐 기술되는 고도화 전략의 도입 당위성**(Rationale)**으로 직결된다. 각 고도화 모듈이 Naïve RAG의 어떤 한계를 해결하기 위해 설계되었는지를 정량적 성능 변화와 함께 추적하는 것이 본 보고서의 핵심 서사 구조다.

---

3. 데이터 엔지니어링 및 구조화

3.1 RFP 데이터 무결성 확보를 위한 메타데이터 보정 전략

문제 인식: "쓰레기 데이터"는 단순 제거로 해결되지 않는다

RAG 시스템의 성능은 검색 모델이나 LLM의 능력 이전에, 입력 데이터의 무결성**(Data Integrity)**에 의해 결정된다. 공공입찰 RFP 데이터는 다음과 같은 구조적 품질 리스크를 내재하고 있었다.

리스크 유형	구체적 사례	**RAG** 파이프라인에 미치는 영향
발주기관명 불일치	동일 기관이 “국민연금공단” / “국민연금관리공단” 등 상이한 명칭으로 등록	메타데이터 필터링 시 동일 기관의 문서를 서로 다른 기관으로 인식
중복 문서 존재	동일 RFP가 HWP와 PDF 두 포맷으로 공존	벡터 DB에 동일 내용이 이중 인덱싱되어 검색 결과 왜곡
메타데이터 필드 결손	사업명, 공고번호 등 핵심 필드의 빈값(NaN) 존재	필터 조건 적용 불가로 전체 DB 대상 검색 발생

이 문제들은 단순히 "더러운 행(row)을 삭제"하는 방식으로는 절대 해결되지 않는다.
보정**(Correction)**이 필요하다.

핵심 보정 메커니즘: `load_metadata_frame` 함수

본 시스템은 `src/bidmate_rag/loaders/metadata_loader.py`에 구현된 `load_metadata_frame` 함수를 통해 다음 5단계의 데이터 무결성 확보 프로세스를 자동화하였다.

1단계: 원본 메타데이터 CSV 로딩 및 타입 정규화

원본 CSV 파일(`data_list.csv`)을 로딩한 직후, 모든 핵심 필드(`공고 번호`, `파일명`, `발주기관`)에 대해 `NaN` 값을 빈 문자열로 치환하고 문자열 타입으로 강제 변환한다. 이 과정을 통해 하위 모듈에서 `NaN` 비교 오류나 타입 불일치로 인한 침묵 실패(Silent Failure)를 원천 차단한다.

2단계: 중복 문서 매핑 테이블 적용

사전에 수동 정제된 `configs/data_quality/duplicates_map.csv` 파일을 로딩한다. 이 파일은 다음 5개의 필수 컬럼을 포함한다.

컬럼명	역할
`duplicate_group_id`	동일 문서 그룹을 식별하는 고유 ID
`source_file`	원본 파일명
`canonical_file`	해당 그룹의 대표(정본) 파일명
`is_duplicate`	중복 여부 (True/False)
`resolved_agency`	보정된 발주기관명

3단계: 발주기관명 일괄 보정

`duplicates_map.csv`에 명시된 `resolved_agency` 값으로 원본 메타데이터의 `발주 기관` 필드를 덮어씌운다. 이를 통해 "국민연금공단"과 "국민연금관리공단"이 동일 기관으로 통합되며, 이후 검색 시 메타데이터 필터의 정합성이 보장된다.

4단계: 중복 문서 인제스트 차단

중복으로 판정된 문서(`is_duplicate == True`)의 경우, 대표 파일이 이미 존재하면 해당 문서의 `ingest_enabled` 플래그를 `False`로 설정하여 벡터 DB 인덱싱에서 자동 제외시킨다. 이를 통해 동일 내용의 이중 인덱싱으로 인한 검색 결과 왜곡을 방지한다.

5단계: 최종 정규화 필드 갱신

보정이 완료된 `resolved_agency` 값을 `발주 기관` 컬럼에 최종 반영하여, 하위 파이프라인(청킹, 검색, 생성) 전체에서 보정된 기관명만 참조되도록 단일 소스(Single Source of Truth)를 확립한다.

데이터 무결성이 **RAG** 성능에 미치는 인과관계

보정 전	보정 후	성능 영향
“A기관” 질문에 B기관 문서 검색	정확히 A기관 문서만 필터링	Hit Rate 상승

동일 내용 2건 중복 검색	정본 1건만 검색	Context Precision 향상
NaN 필드로 필터 무효화	빈 문자열로 정규화 완료	침묵 실패 제거

이 메타데이터 보정 전략은 화려한 AI 기술이 아니다. 그러나 이 지루하고 수작업적인 데이터 품질 확보 없이는, 아무리 정교한 검색 모델도 제 성능을 발휘할 수 없다. 본 프로젝트가 데이터 무결성을 최우선 과제로 설정한 이유가 여기에 있다.

3.2 B2G 문서 구조 보존형 전처리 파이프라인

도입 배경: 왜 범용 PDF 추출기로는 부족한가?

공공입찰 RFP 문서는 일반적인 PDF 문서와 근본적으로 다른 구조적 특성을 갖는다. 대부분의 원본 파일이 **HWP(**한글**)** 포맷으로 작성되어 있으며, 내부에는 복잡한 다단 구조, 중첩된 표(Table), 로마자/아라비아 숫자 혼합 목차, 그리고 수십 페이지에 걸친 평가 배점표가 포함되어 있다.

과거의 단순 텍스트 추출 방식(PDFMiner, PyPDF2 등 범용 도구)은 이러한 B2G(Business-to-Government) 문서에서 다음과 같은 치명적 결함을 보였다.

파괴 유형	구체적 현상	**LLM**에 미치는 영향
표 구조 해체	3열 × 20행 표가 연속된 문자열로 풀어짐	숫자·항목 대응관계 완전 소실
다단 레이아웃 붕괴	좌측 요구사항과 우측 배점이 뒤섞임	엉뚱한 요구사항에 엉뚱한 배점 매칭
목차 계층 소실	"Ⅱ. 제안 평가 기준 > 1. 기술 평가" 구조가 평문화	청크가 자신의 소속 섹션을 알 수 없음
유니코드 깨짐	PUA(Private Use Area) 문자 잔존	검색 인덱싱 시 노이즈 발생

이러한 '쓰레기 데이터의 유입(Garbage In)'은 곧바로 LLM의 '환각 현상(Hallucination)'으로 직결된다. 파싱 품질이 곧 **RAG**의 천장인 것이다.

Kordoc 주력 엔진 기반의 구조 복원 아키텍처

이 문제를 근본적으로 해결하기 위해, 본 시스템은 **Kordoc**을 주력 파싱 엔진으로 채택하였다. Kordoc은 Node.js 기반의 한국어 문서 전문 파서로, HWP와 PDF 파일 모두에 대해 마크다운(Markdown) 형태의 구조화된 출력을 생성한다.

파싱 파이프라인 처리 흐름

본 시스템의 파싱 파이프라인은 다음과 같은 단계로 구성된다.

1단계**:** 원본 포맷 인식 및 **Kordoc** 엔진 호출

문서가 유입되면 시스템은 파일의 확장자(`hwp`, `pdf`, `docx`)를 식별한 뒤, Kordoc CLI를 subprocess로 호출한다. Kordoc은 문서의 물리적 레이아웃을 분석하여, 표(Table)는 마크다운 표 문법(`| col1 | col2 |`)으로, 목차는 마크다운 헤딩(`#`, `##`, `###`)으로, 본문은 일반 텍스트로 변환한다.

2단계**:** 파싱 실패 시 **Fallback** 경로 활성화

Kordoc이 특정 문서에서 텍스트 추출에 실패하거나 빈 결과를 반환할 경우, 시스템은 즉시 보조 엔진(`hwp-hwpx-parser`)으로 자동 전환하여 파싱을 재시도한다. 이 이중 안전장치를 통해 단일 파서의 장애가 전체 파이프라인의 중단으로 이어지지 않도록 설계하였다.

파싱 성능 수치

지표	수치
총 처리 문서 수	101건 (HWP 96, PDF 4, DOCX 1)
문서당 평균 처리 시간	약 0.9초
평균 추출 텍스트 길이	106,566자/문서
CSV 대비 텍스트 추출 증가율	평균 28배

6종 노이즈 정제 파이프라인 **(Cleaner)**

Kordoc으로 추출된 마크다운 텍스트는 아직 '깨끗한' 상태가 아니다. HWP 포맷 특유의 잔존 노이즈를 제거하기 위해, `src/bidmate_rag/preprocessing/cleaner.py`에 구현된 **6**종 순차 정제 파이프라인을 통과시킨다.

이 정제 파이프라인은 다음과 같은 엄격한 순서로 실행된다. 순서가 바뀌면 정제 결과가 달라지므로, 실행 순서 자체가 하나의 설계 의사결정이다.

순서	정제 함수	제거 대상	제거량
①	`clean_kordoc_warnings`	Kordoc 엔진이 삽입한 Warning 라인	전량
②	`clean_br_tags`	HTML ` ` 태그 → 실제 줄바꿈 변환	102,922개
③	`remove_empty_html_tables` / `remove_toc_html_tables`	빈 HTML 테이블 및 목차 역할의 HTML 테이블 제거	—
④	`remove_text_toc_block`	일반 텍스트 형태의 목차 블록 제거	—
⑤	`normalize_section_box_tables`	섹션 제목처럼 생긴 1행 테이블을 정상 마크다운 헤딩으로 변환	—
⑥	`clean_duplicate_table_cells` / `clean_broken_chars` / `clean_whitespace`	중복 셀 축소, PUA 문자·NBSP 제거, 연속 공백 정규화	6,583 중복셀 / 2,831 PUA문자

특히 `normalize_section_box_tables` 함수는 매우 정교한 로직을 포함한다. HWP에서 변환된 마크다운에는 종종 "Ⅱ. 제안 평가 기준"과 같은 목차 제목이 1행 테이블(`| Ⅱ | 제안 평가 기준 |`) 형태로 출력되는 경우가 있다. 이 함수는 이러한 패턴을 감지하여 로마 숫자 마커는 `##` 헤딩으로, 아라비아 숫자 마커는 `###` 헤딩으로 정규화함으로써, 이후 청킹 단계에서 올바른 섹션 분할이 이루어지도록 보장한다.

정제 전후 텍스트 크기 변화

단계	문서당 평균 글자수	변화량
파싱 직후 (정제 전)	106,566자	—

정제 완료 후	99,105자	△7,461자 (약 7% 감소)

감소분 7%는 순수 노이즈에 해당하며, 이 노이즈가 제거되지 않았다면 벡터 인덱싱 시 불필요한 차원을 점유하여 검색 정밀도를 오염시켰을 것이다.

3.3 문자열 분할과 의미 기반 동적 청킹 스위칭 메커니즘

청킹이란 무엇이며, 왜 이토록 중요한가

정제된 마크다운 문서는 문서당 평균 99,105자에 달한다. 이 방대한 텍스트를 LLM의 컨텍스트 윈도우에 통째로 넣을 수는 없다. 따라서 문서를 의미 단위의 작은 조각(Chunk)으로 분할하고, 질문과 가장 관련 있는 조각만 골라서 LLM에 전달해야 한다. 이 분할 과정의 품질이 곧 검색 품질을 결정하며, 검색 품질이 답변 품질을 결정한다.

잘못된 청킹의 3가지 재앙:

1. 너무 작게 자르면 — 문장 앞뒤의 문맥이 잘려나가 대명사가 지칭하는 대상을 LLM이 알 수 없게 된다
2. 너무 크게 자르면 — 포함되는 노이즈가 증가하여 LLM의 집중도가 떨어지고, 검색 시 정밀도가 하락한다
3. 논리적 경계를 무시하고 자르면 — "요구사항 A"의 뒷부분과 "요구사항 B"의 앞부분이 하나의 청크에 섞여, 검색 결과가 어느 쪽도 아닌 혼합 텍스트가 된다

3단 계층형 청킹 전략

본 시스템은 이 세 가지 재앙을 동시에 방지하기 위해, 3단 계층형 청킹 전략을 설계하였다.

제1단: 마크다운 헤더 기반의 의미론적 계층화 분할

추출된 마크다운 구조에서 `#`(대분류) 헤더를 기준으로 텍스트를 1차 그룹화한다. 이 과정에서 `MarkdownHeaderTextSplitter`를 사용하여, 각 섹션이 자신이 속한 목차 정보(`h1` 헤더)를 메타데이터로 보존한다.

그러나 단순히 헤더마다 자르면 지나치게 작은 섹션이 발생할 수 있다. 예를 들어 " I . 사업 개요"라는 섹션이 200자밖에 되지 않는 경우가 있다. 이런 소형 섹션은 독립 청크로 만들기에는 문맥이 부족하다.

이를 해결하기 위해 최소 섹션 크기(**min_section_size = 500**자) 임계값을 설정하였다. 500자 미만의 섹션은 자동으로 다음 섹션과 병합되어, 의미적으로 충분한 크기의 텍스트 블록을 형성한다.

제**2**단**:** 재귀적 문자열 분할 **(Recursive Character Text Splitting)**

헤더 기준으로 분류된 후에도 여전히 크기가 큰 섹션(1,000자 초과)은 2차 분할을 수행한다. 이때 `RecursiveCharacterTextSplitter`가 다음과 같은 계층적 분리자 우선순위를 따른다.

우선순위	분리자	의미
1순위	`\n\n` (빈 줄)	단락 경계 — 가장 자연스러운 분할 지점
2순위	`\n` (줄바꿈)	문장 경계
3순위	`. ` (마침표+공백)	문장 내부 분할
4순위	(공백)	단어 경계 분할
5순위	`""` (빈 문자열)	문자 단위 분할 (최후의 수단)

시스템은 1순위 분리자로 먼저 분할을 시도하고, 그래도 청크 크기가 초과하면 2순위로 넘어가는 식으로 점진적으로 세밀한 분할을 수행한다. 이 재귀적 접근을 통해, 가능한 한 의미 단위가 보존되는 가장 큰 경계에서 분할이 이루어진다.

제**3**단**:** 의미 기반 분할 스위치 **(Semantic Chunking)**

텍스트의 성격이 고난도 복합 논리일 경우, 단순 문자 수 기준 분할로는 한계가 있다. 예를 들어, 하나의 긴 단락 안에서 주제가 "보안 요구사항"에서 "성능 요구사항"으로 전환되는데 문자 수로는 구분이 불가능한 경우가 있다.

이를 위해 본 시스템은 `SemanticChunker` 모드를 지원한다. 이 모드에서는 `text-embedding-3-small` 임베딩 모델을 사용하여 연속된 문장들의 임베딩 벡터를 계산한 뒤, 인접 문장 간의 코사인 유사도가 급격히 하락하는 지점을 수학적 백분위(Percentile) 기반으로 탐지한다.

파라미터	설정값	의미

breakpoint_threshold_type	percentile	유사도 하락 임계값을 백분위로 판단
breakpoint_threshold_amount	95.0	상위 95% 이상의 유사도 하락만 경계로 인정

즉, 문장 간 유사도 하락이 전체 분포에서 상위 5%에 해당할 만큼 극단적일 때만 분할이 발생한다. 이 보수적인 임계값을 통해 과도한 분할(Over-segmentation)을 방지하면서도, 진정한 주제 전환 지점을 포착할 수 있다.

시맨틱 청킹으로 생성된 청크가 여전히 `chunk_size`를 초과하는 경우, 재귀적 분할기가 후처리(post-processing)로 추가 분할을 수행하여 최종 크기 제한을 보장한다.

테이블 보존형 분할: 표 구조의 무결성 확보

RFP 문서에서 표(Table)는 단순한 데이터가 아니라 평가 기준, 배점, 일정, 예산 등 핵심 합목적 정보를 담고 있다. 표를 일반 텍스트와 동일하게 취급하여 중간에서 잘라버리면, 열(Column) 헤더와 데이터 행의 대응 관계가 파괴된다.

본 시스템은 `is_table_block` 함수를 통해 텍스트 블록이 마크다운 표인지를 판별한다. 파이프(`|`) 기호로 시작하는 줄이 전체의 50%를 초과하면 해당 블록을 표로 분류한다.

표로 분류된 블록이 최대 테이블 크기(`max_table_size = 1,500`자)를 초과하는 경우, `split_table_with_headers` 함수가 작동하여 다음과 같은 정교한 분할을 수행한다.

1. 테이블의 헤더 행(컬럼명 + 구분선)을 추출한다
2. 데이터 행을 순차적으로 누적하며, `max_table_size` 이내인지 확인한다
3. 초과 시 현재까지의 행으로 하나의 청크를 확정하고, 헤더를 다시 붙여 새 청크를 시작한다

이 "헤더 반복 삽입(Header Carry-over)" 전략을 통해, 분할된 테이블 청크 각각이 독립적으로 해석 가능한 완전한 표 구조를 유지하게 된다.

시스템 파라미터 의사결정: 왜 1,000자인가

프리셋	chunk_size	chunk_overlap	min_section_size	max_table_size	적합 시나리오

small	500	100	300	1,000	팩토이드/단답형 질문
medium (채택)	1,000	150	500	1,500	범용 (베이스라인)
large	1,500	200	700	2,000	분석형/비교 질문

Chunk Size = 1,000자를 채택한 근거**:**

- 100건의 RFP 문서를 분석한 결과, 하나의 세부 요구사항은 평균 600~1,200자 범위에 분포
- 1,000자는 일반적인 제안요청서의 **1~2**개 세부 요구사항을 온전히 담아낼 수 있는 황금비율
- 이보다 작으면(예: 500자) 문맥 단절이 빈번하고, 이보다 크면(예: 1,500자) 노이즈 비율이 증가

Chunk Overlap = 150자를 설정한 근거**:**

- 분리된 청크 간에 150자의 겹치는 영역을 두어, 경계선에서 잘린 문장의 연결고리를 제공
- Overlap이 너무 작으면(예: 50자) 문장 절단 빈도가 상승하고, 너무 크면(예: 300자) 중복 인덱싱으로 저장 공간이 낭비

최종 청킹 결과 통계 (4가지 전략 비교)

청킹 전략	총 청크 수	평균 크기(자)	중앙값(자)	테이블 청크 수	테이블 비율(%)
500 / 100 (소형)	9,252개	1,285자	1,444자	2,440개	26.3%
1000 / 150 (베이스라인)	7,943개	1,493자	1,469자	1,131개	14.2%
1500 / 200 (대형)	7,556개	1,568자	1,480자	744개	9.8%
Semantic Percentile	7,814개	1,501자	1,471자	1,002개	12.8%

- 데이터 클리닝 및 정제 작업 이후 전체 청크 수가 크게 증가하였으며, 문서 구조 파악을 위해 1000/150을 최종 베이스라인으로 채택하였습니다.

데이터 클리닝 전후 효과 분석 **(1000/150** 기준**)**

Kordoc 엔진의 노이즈 처리 및 메타데이터 정규화 등 클리닝 작업을 파이프라인에 추가한 후, 베이스라인(1000/150)에서 극적인 성능 향상이 관찰되었습니다. 이는 최적의 청킹이나 고도의 검색 모델보다 입력 데이터의 무결성**(Data Integrity)** 확보가 **RAG** 성능에 가장 지대한 영향을 미침을 입증합니다.

품질 지표	클리닝 이전	클리닝 이후	개선 폭
Hit Rate@5	0.9733	1.0000	↑ 0.0267 (+100% 도달)
MRR	0.9578	0.9792	↑ 0.0214
Faithfulness	0.6094	0.6906	↑ 0.0812
Answer Relevance	0.6344	0.7375	↑ 0.1031

데이터 정제로 인해 파싱 실패로 누락되었던 텍스트가 컨텍스트로 온전히 제공되면서, 충실도(Faithfulness)와 답변 적합성(Answer Relevance)이 약 10%p 안팎으로 크게 뛰었습니다.

3.4 메타데이터 워터마크 주입을 통한 도메인 컨텍스트 확립

문제**:** 문맥을 상실한 떠돌이 문장

청킹이 완료되면 14,035개의 텍스트 조각이 생성된다. 그러나 이 조각들은 원본 문서로부터 떨어져 나온 순간, 자신이 어떤 기관의 어떤 사업에 대한 내용인지를 알 수 없게 된다**.** 예를 들어 다음과 같은 청크가 있다고 하자:

“제안업체는 1,000TPS 이상의 처리 성능을 보장해야 하며, 24시간 무중단 운영이 가능한 아키텍처를 설계해야 한다.”

이 문장만으로는 어떤 기관의 어떤 사업에 대한 요구사항인지 전혀 알 수 없다. 검색 시 "국민연금공단의 성능 요구사항"을 물었을 때, 이 청크가 실제로 국민연금공단 문서에서 나온 것인지, 다른 기관의 유사한 요구사항인지 구별이 불가능하다.

해결: 모든 청크에 도메인 식별 워터마크를 각인

본 시스템은 `src/bidmate_rag/preprocessing/chunker.py`의 `_meta_prefix` 함수를 통해, 모든 청크의 본문 앞에 메타데이터 접두사**(Prefix)**를 강제 주입한다.

접두사 형식:

`[발주기관: 국민연금공단 | 사업명: 차세대 정보시스템 구축]`

이 접두사가 주입된 텍스트는 `text_with_meta` 필드에 저장되며, 임베딩 벡터 생성 시 이 접두사가 포함된 텍스트가 입력으로 사용된다. 즉, 임베딩 모델은 본문의 의미뿐 아니라 기관명과 사업명의 의미까지 함께 벡터화한다.

워터마크 주입이 검색에 미치는 효과

시나리오	워터마크 없음	워터마크 있음
"국민연금공단 성능 요구사항" 검색	유사 문구를 가진 타 기관 문서도 함께 반환	국민연금공단 문서의 청크가 우선 순위 상승
"AI 기반 사업의 보안 요건" 검색	AI 키워드만 매칭, 기관 무관	기술스택에 AI가 포함된 사업의 청크가 부스팅

추가 메타데이터 보강: 도메인 분류 체계

단순히 기관명과 사업명만 부착하는 것이 아니라, 본 시스템은 각 문서에 대해 다음과 같은 분류 속성을 자동 산출하여 메타데이터에 포함시킨다.

기관유형 자동 분류 (`classify_agency` 함수):

기관유형	판별 키워드 예시
중앙행정기관	"부", "처", "청", "위원회"
대학교	"대학", "학교"
연구기관	"연구원", "연구소", "과학"
공기업/준정부기관	"공사", "공단", "진흥원"

지방자치단체	“특별시”, “광역시”, “도”
협회/재단	“협회”, “재단”, “센터”

사업도메인 자동 분류 (`classify_domain` 함수):

도메인	판별 키워드 예시
교육/학습	“교육”, “이러닝”, “LMS”
안전/재난	“안전”, “재난”, “관제”
웹/포털	“홈페이지”, “포털”, “플랫폼”
AI/데이터	“AI”, “인공지능”, “빅데이터”
의료/바이오	“의료”, “건강”, “바이오”

기술스택 자동 추출 (`extract_tech_stack` 함수):

기술 카테고리	판별 키워드
AI	“AI”, “인공지능”, “머신러닝”, “딥러닝”
클라우드	“클라우드”, “cloud”, “SaaS”
IoT	“IoT”, “센서”, “SCADA”
GIS	“GIS”, “지도”, “공간정보”
모바일	“모바일앱”, “모바일 앱”

이렇게 생성된 12개 메타데이터 필드가 각 청크에 부착됨으로써, 검색 시 `where` 필터를 통해 **“대학교 사업만”, “AI 관련 사업만”, “2024년 사업만”** 등의 정밀한 범위 축소가 가능해진다.

최종 메타데이터 스키마 일람

필드명	출처	용도
사업명	CSV 원본	출처 표시, 필터링
발주 기관	CSV 원본 → 보정	기관별 필터링
공고 번호	CSV 원본	문서 고유 식별
사업금액	CSV 원본	금액 범위 필터
기관유형	규칙 기반 자동 분류	“중앙행정기관만” 필터
사업도메인	사업명+본문 키워드 분석	“교육 관련” 필터
기술스택	본문 키워드 추출	“AI 관련” 필터
공개연도	공개일자 변환	“2024년” 필터
section	청킹 시 헤딩 보존	섹션 힌트 부스팅
content_type	표/텍스트 자동 구분	테이블 부스팅
text_with_meta	접두사 + 본문 결합	임베딩 입력
resolved_agency	보정된 기관명	검색 정합성 확보

이로써 모든 청크는 단순한 텍스트 조각이 아니라, 자신의 출처, 소속 기관, 사업 도메인, 기술 분야, 문서 내 위치까지 완벽하게 기억하는 컨텍스트 인식형 데이터 유닛으로 변모한다. 이 설계가 이후 4장의 벡터 DB 구축과 5장의 메타데이터 기반 하이브리드 검색의 기술적 토대가 된다.

4. RAG 시스템 아키텍처 (Scenario Design)

4.1 시나리오 A: 로컬 거대 언어 모델 최적화 — Native FP16 모델 원천 지능 보존 전략

설계 철학: 왜 로컬 LLM인가

공공입찰 RFP 문서에는 사업 예산, 기관 내부 구조, 입찰 자격 조건 등 민감한 행정 정보가 다수 포함되어 있다. 기업 보안 정책상 이러한 문서 내용이 외부 API(OpenAI, Google 등)의 서버로 전송되는 것을 허용하지 않는 경우가 빈번하다. 시나리오 A는 이러한 프라이빗 환경**(Air-gapped Environment)**을 전제로, 외부 네트워크 접근 없이 조직 내부의 GPU 서버에서 모든 추론을 완결하는 아키텍처를 설계하였다.

그러나 로컬 LLM 운영에는 다음과 같은 근본적인 엔지니어링 제약이 존재한다.

제약 요소	구체적 문제	영향
VRAM 한계	NVIDIA L4 GPU의 24GB VRAM으로는 7B 파라미터 모델의 FP32 로딩 불가 (약 28GB 필요)	양자화(Quantization) 필수
추론 속도	로컬 GPU에서의 토큰 생성 속도는 클라우드 API 대비 2~5배 느림	사용자 경험 저하
한국어 성능	대부분의 오픈소스 LLM은 영문 중심 학습, 한국어 도메인 특화 부족	답변 품질 하락
프롬프트 길이	컨텍스트 윈도우가 제한적 (4K~8K 토큰)	검색된 청크의 투입량 제한

대상 모델 선정: Gemma-4-E4B-it 및 Qwen2.5-7B

시나리오 A에서는 두 가지 로컬 오픈소스 LLM을 대상으로 비교 실험을 수행하였다.

모델 비교 매트릭스

속성	**Gemma-4-E4B-it**	**Qwen2.5-7B-Instruct**
개발사	Google DeepMind	Alibaba Cloud

파라미터 규모	4B 클래스	7B
학습 데이터 언어	다국어 (한국어 포함)	다국어 (한국어 최적화)
라이선스	Gemma License	Apache 2.0
추론 프레임워크	vLLM / Ollama	HuggingFace Transformers
프로바이더 설정	`configs/providers/scenario_a/`	`configs/providers/scenario_a/llms/qwen2.5-7b.yaml`
주요 강점	경량화, 빠른 추론	한국어 도메인 이해도

Qwen2.5-7B 선정 근거

Qwen2.5-7B-Instruct는 Alibaba Cloud의 Qwen 시리즈 중 한국어 최적화 모델로 평가받고 있다. 시스템의 `configs/providers/scenario_a/llms/qwen2.5-7b.yaml` 설정 파일에 다음과 같이 등록되어 있다.

설정 키	값
`provider`	huggingface
`model`	Qwen/Qwen2.5-7B-Instruct
`description`	Qwen2.5 7B 한국어 최적화 모델

이 모델은 HuggingFace 생태계를 통해 로딩되며, 시나리오 A의 파이프라인에서 `build_llm_provider` 함수가 프로바이더 설정을 읽어 자동으로 인스턴스를 생성한다.

VRAM 다이어트 전략: 양자화와 추론 최적화

24GB VRAM이라는 물리적 제약 아래에서 7B 파라미터 모델을 구동하기 위해, 본 시스템은 다음과 같은 다단계 최적화 전략을 적용하였다.

1단계: 4-bit Quantization (bitsandbytes)

모델의 가중치를 FP32(32비트) 또는 FP16(16비트)에서 INT4(4비트)로 양자화한다. 이를 통해 VRAM 사용량이 약 1/4~1/8로 축소된다.

정밀도	**7B 모델 예상 VRAM**	**L4 GPU (24GB)** 적합성
FP32	~28GB	❌ 초과
FP16	~14GB	⚠️ 빠듯
INT8	~7GB	✅ 여유
INT4	~3.5GB	✅ 충분

단, 양자화는 필연적으로 모델의 원천 지능**(Native Intelligence)**을 훼손한다. 특히 4-bit 양자화 시 한국어 조사 처리, 복합 논리 추론, 숫자 정확성에서 성능 저하가 관찰될 수 있다. 이것이 시나리오 A의 본질적 트레이드오프(Trade-off)다.

2단계: vLLM / Ollama 기반 추론 서버 분리

모델 로딩과 추론 요청을 분리하여, 한 번 로딩된 모델이 메모리에 상주하면서 다수의 질의를 효율적으로 처리하도록 구성한다. 이를 통해 매 질의마다 모델을 재로딩하는 비효율을 제거하였다.

3단계: PEFT(Parameter-Efficient Fine-Tuning) 준비

`pyproject.toml`의 `ml` 의존성 그룹에 `peft`, `trl`, `datasets` 등의 파인튜닝 라이브러리가 포함되어 있으며, `scripts/train_peft.py` 스크립트가 이미 구현되어 있다. 이를 통해 향후 RFP 도메인 특화 파인튜닝을 수행할 수 있는 기반을 마련하였다.

시나리오 A의 파이프라인 흐름

시나리오 A에서의 전체 데이터 흐름은 다음과 같다.

단계	처리 내용	실행 환경
① 질문 입력	사용자 질문 수신	로컬 서버
② 임베딩	BAAI/bge-m3 임베딩 벡터 생성	로컬 GPU

③ 검색	ChromaDB에서 Top-K 청크 검색	로컬 스토리지
④ 컨텍스트 조립	검색 결과 + 시스템 프롬프트 결합	CPU
⑤ 추론	Gemma-4-E4B-it 또는 Qwen2.5-7B로 답변 생성	로컬 GPU
⑥ 응답 반환	생성된 답변 출력	로컬 서버

핵심 설계 원칙**:** 전 과정에서 단 한 바이트도 외부 서버로 유출되지 않는다**.**

4.2 시나리오 **B:** 고성능 상용 **API** 파이프라인 **—** 품질 천장 확인용 레퍼런스 아키텍처

설계 철학**:** 성능의 절대 상한선은 어디인가

시나리오 B는 시나리오 A와 정반대의 설계 철학을 따른다. 비용과 프라이버시를 양보하는 대신, 현존하는 최고 수준의 **LLM** 성능을 내 파이프라인에 투입하여 **RAG** 시스템이 도달할 수 있는 품질 천장**(Ceiling)**을 확인하는 것이 목적이다.

이 레퍼런스 모델과 시나리오 A 로컬 모델의 성능 간극(Performance Gap)을 정량적으로 측정함으로써, 다음과 같은 전략적 의사결정에 활용한다.

의사결정 항목	판단 기준
로컬 모델 품질이 충분한가?	시나리오 B 대비 Faithfulness/Relevance 차이가 10% 이내인가?
파인튜닝에 투자할 가치가 있는가?	성능 간극이 파인튜닝으로 좁힐 수 있는 범위인가?
프로덕션 배포 모델 선택	비용 대비 품질(Cost-Quality Ratio) 최적점은 어디인가?

시나리오 **B** 모델 구성

속성	설정값
Generation LLM	GPT-5-mini / GPT-5 / GPT-5-nano
임베딩 모델	BAAI/bge-m3 (시나리오 간 통일)
Judge 모델	gpt-4o-mini
API 엔드포인트	OpenAI API (cloud)
과금 방식	토큰 단위 종량제

시나리오 B의 핵심은 임베딩 모델을 시나리오 **A**와 동일하게 고정함으로써, 검색(Retrieval) 단계의 변수를 제거하고 순수하게 생성**(Generation)** 단계의 **LLM** 성능 차이만을 분리 측정할 수 있도록 설계한 것이다. 실험 설정에서 `mode: generation_only`로 지정하면 동일 인덱스(ChromaDB 컬렉션)를 공유하며 LLM만 교체하여 비교할 수 있다.

두 시나리오의 전략적 대조

비교 축	시나리오 **A (**로컬 **LLM)**	시나리오 **B (**상용 **API)**
데이터 보안	✅ 완전 내부 처리	⚠️ 외부 API 전송
응답 품질	양자화로 인한 성능 저하 우려	최고 수준의 추론 능력
Latency	GPU 성능에 종속 (5~30초)	네트워크 + API 처리 (2~8초)
운영 비용	GPU 서버 유지 비용 (고정비)	토큰당 과금 (변동비)
확장성	GPU 추가 필요	API 호출 수 증가로 즉시 확장
한국어 특화	모델 선택에 따라 가변	GPT-5 시리즈의 다국어 지원
파인튜닝	PEFT 적용 가능	불가 (API 모델)

4.3 초정밀 벡터 데이터베이스(Vector DB) 메타 필터 레이어 구축

ChromaDB를 선택한 엔지니어링 근거

벡터 데이터베이스의 선택은 RAG 시스템의 검색 성능과 운영 효율성에 직접적인 영향을 미친다. 본 시스템은 다음과 같은 이유로 **ChromaDB**를 채택하였다.

선정 기준	**ChromaDB**의 강점
로컬 퍼시스턴스	`PersistentClient` 지원으로 GPU 서버 로컬 디스크에 직접 저장
메타데이터 필터링	`where` 절을 통한 구조화된 필터 native 지원
Python 생태계	LangChain, HuggingFace와의 원활한 통합
경량성	별도 서버 프로세스 없이 인프로세스(in-process) 실행
코사인 유사도	기본 거리 함수로 코사인 유사도 지원

벡터 인덱스 구축 프로세스

3장에서 생성된 14,035개의 청크는 다음과 같은 프로세스를 거쳐 벡터 DB에 인덱싱된다.

단계	처리 내용	수치
① 최소 크기 필터링	50자 미만 청크 제외	14,035 → 13,951개
② 임베딩 벡터 생성	`text_with_meta` 필드를 BAAI/bge-m3로 인코딩	1,536차원 벡터
③ 메타데이터 동시 저장	12개 메타데이터 필드를 벡터와 함께 저장	—
④ 퍼시스턴스	로컬 디스크에 영구 저장	384MB

임베딩 비용: 약 $0.15 (207원) — text-embedding-3-small 기준

컬렉션 격리 전략: 실험 간 데이터 오염 방지

본 시스템은 동시에 수십 개의 실험을 병렬로 수행할 수 있도록, 실험 모드에 따른 컬렉션 격리 전략을 구현하였다.

실험 모드	컬렉션 관리	이유
`full_rag`	`{실험명}-{프로바이더}-{모델}` 형태로 격리	청킹 파라미터가 다르면 인덱스가 달라야 함
`generation_only`	공용 컬렉션 공유	동일 인덱스에서 LLM만 교체하여 비교
`ad-hoc`	`bidmate-{프로바이더}-{모델}` 기본 패턴	일회성 질의용

이 격리 전략의 핵심은 청킹 파라미터가 변경되면 반드시 새로운 컬렉션을 생성하여 **Stale Chunk(**오래된 청크 잔존**)** 문제를 차단하는 것이다. `chunk_size=500`으로 생성된 인덱스와 `chunk_size=1000`으로 생성된 인덱스가 혼합되면, 검색 결과의 일관성이 파괴된다.

Where Clause: 메타데이터 기반 하드 필터

ChromaDB의 `where` 필터는 벡터 유사도 검색 이전에 물리적으로 검색 범위를 축소하는 하드 필터(Hard Filter)다. 이는 검색 정밀도와 속도 모두에 극적인 영향을 미친다.

필터 유형	사용 예시	효과
기관 필터	`{"발주 기관": "국민연금공단"}`	13,951개 → 해당 기관 문서의 청크만 검색
도메인 필터	`{"사업도메인": "교육/학습"}`	교육 관련 사업의 청크만 검색
기관유형 필터	`{"기관유형": "대학교"}`	대학교 발주 사업만 검색
범위 필터	`{"사업 금액": {"$gte": 100000000}}`	1억 이상 사업만 검색
연도 필터	`{"공개연도": 2024}`	2024년 공개 사업만 검색

복합 필터	`{"발주 기관": {"$in": ["A기관", "B기관"]}}`	A 또는 B 기관 문서만 검색

이 하드 필터가 없다면, "국민연금공단의 보안 요구사항은?"이라는 질문에 대해 시스템은 13,951개 전체 청크를 코사인 유사도로 비교해야 하며, 보안 관련 키워드가 유사한 타 기관의 문서가 상위권에 침입할 수 있다. 하드 필터 적용으로 검색 대상이 해당 기관의 수백 개 청크로 축소되면, 검색 정밀도와 속도가 동시에 향상된다.

---

5. 검색 및 생성 고도화 전략

5.1 한국어 도메인 특화 임베딩(BAAI/bge-m3) 모델 성능 평가

임베딩 모델 선정의 전략적 중요성

RAG 시스템에서 임베딩 모델은 질문과 문서 청크를 동일한 벡터 공간에 매핑하는 번역기 역할을 한다. 이 번역기의 품질이 낮으면, 아무리 정교한 검색 알고리즘이나 Reranker를 적용해도 근본적으로 정답 청크를 찾을 수 없다.

공공입찰 RFP 도메인은 일반적인 한국어 텍스트와 현저히 다른 어휘적 특성을 갖는다.

특성	일반 한국어 텍스트	**RFP** 도메인 텍스트
주요 어휘	일상 대화, 뉴스 기사	“지체상금률”, “수의계약”, “기술성평가”
숫자 중요도	보조적	핵심적 (예산, 기간, 배점)
고유명사 밀도	낮음	높음 (기관명, 시스템명, 법률 조항)

표 내부 텍스트	드묾	매우 빈번

BAAI/bge-m3 채택 근거

본 시스템은 모든 실험에서 임베딩 모델을 **BAAI/bge-m3**로 고정하였다. 이 모델은 실험 설정 파일(`configs/experiments/gemma_fixed_eval.yaml`)에서 다음과 같이 명시되어 있다.

설정 키	값
`embedding_model`	BAAI/bge-m3
`embedding_provider`	huggingface

BGE-M3(Multilingual, Multi-Functionality, Multi-Granularity)는 다음과 같은 이유로 선정되었다.

선정 기준	**BGE-M3**의 강점
다국어 지원	100개 이상 언어 학습, 한국어 포함
Dense + Sparse 동시 지원	하이브리드 검색에 단일 모델로 대응 가능
벡터 차원	1,536차원 — 의미 표현 해상도가 높음
문맥 윈도우	8,192 토큰 — 긴 청크도 온전히 임베딩 가능
오픈소스	로컬 구동 가능 — 시나리오 A와 호환

임베딩 입력 전략: 메타데이터 접두사 포함 여부

3장에서 설명한 바와 같이, 각 청크는 두 가지 텍스트 필드를 보유한다.

필드	내용	용도
`text`	순수 본문 텍스트	LLM 컨텍스트 전달용
`text_with_meta`	`[발주기관: X | 사업명: Y]\n본문`	임베딩 벡터 생성용

임베딩 시 `text_with_meta`를 입력으로 사용함으로써, 벡터 공간 내에서 동일 기관의 청크들이 자연스럽게 클러스터를 형성하게 된다. 이 접두사는 전체 텍스트의 약 5%를 차지하며, 임베딩 모델이 기관/사업 정보를 의미적으로 인코딩하는 데 충분한 신호를 제공한다.

5.2 의도 추론 기반 적응형 하이브리드 검색 오케스트레이션 (Retriever)

왜 단순 벡터 검색으로는 부족한가

2장에서 확인한 Naïve RAG의 한계 중 가장 치명적인 것은 기관 혼동과 고유명사 검색 실패였다. Dense 검색만으로는 "국민연금공단의 2024년 이러닝 사업 예산"이라는 질문에서 '국민연금공단'과 '2024년'이라는 정확한 식별자를 구분하지 못하고, 유사한 맥락의 타 기관 데이터를 가져올 수 있다.

이 문제를 체계적으로 해결하기 위해, 본 시스템의 Retriever는 단순한 검색 모듈이 아니라 7단계의 의사결정 파이프라인을 오케스트레이션하는 지휘자로 설계되었다.

검색 오케스트레이션 7단계 상세 분석

제1단: 내재 의도 스캐닝 및 메타데이터 필터 자동 추출

사용자 질문이 입력되면, `src/bidmate_rag/retrieval/filters.py`의 `extract_metadata_filters` 함수가 즉시 작동하여 질문 내부의 의도를 해부한다. 이 함수는 다음과 같은 계층적 필터 추출 로직을 실행한다.

기관명 추출 우선: 먼저 전체 발주기관 목록(`agency_list`)과 질문을 교차 대조하여 기관명을 추출한다. 기관이 1개 감지되면 즉시 `{"발주 기관": "감지된기관"}` 필터를 반환한다. 기관이 2개 이상 감지되면 비교형 질문으로 판단하여 단일 필터를 적용하지 않고 팬아웃(Fan-out) 검색으로 분기시킨다.

도메인 키워드 추출: 기관명이 감지되지 않으면, 질문에서 도메인 키워드를 스캔한다. 12개 도메인 카테고리(교육/학습, 안전/재난, 웹/포털, AI/데이터 등)에 대한 판별 키워드 사전이 정의되어 있으며, 매칭되는 도메인이 있으면 `{"사업도메인": "매칭도메인"}`으로 필터를 생성한다.

기관유형 추출: 도메인도 매칭되지 않으면, “대학교”, “공기업”, “지자체” 등의 기관유형 키워드를 탐색한다.

범위 필터 병합: 별도로 `extract_range_filters` 함수가 “10억 이상”, “2024년” 등의 금액 범위와 연도 조건을 정규표현식으로 추출하여 Where 절에 추가한다.

회피 키워드 감지: 질문에 “다른 기관”, “외에”, “말고” 등의 키워드가 포함되면, 현재 문맥의 기관을 의도적으로 배제하기 위해 필터를 적용하지 않는다. 이 섬세한 예외 처리가 없으면, "국민연금공단 말고 다른 기관의 사례는?"이라는 질문에 오히려 국민연금공단만 필터링하는 역설이 발생한다.

제2단: 멀티턴 대화 기반 맥락 상속 **(History-aware Query Rewrite)**

후속 질문에서 "그 사업의 예산은?"과 같이 대명사가 사용되면, 시스템은 이전 대화 맥락에서 지시 대상을 복원해야 한다. `src/bidmate_rag/retrieval/multiturn.py`에 구현된 쿼리 재작성 엔진이 이 역할을 수행한다.

재작성 엔진은 3가지 모드를 지원한다.

모드	작동 방식	용도
`llm_with_rule_fallback`	LLM으로 재작성 시도 → 실패 시 규칙 기반 폴백	기본 설정
`rule_only`	LLM 없이 규칙 기반만 사용	비용 절감, 로컬 환경
`llm_only`	LLM 재작성만 사용	최고 품질 추구

LLM 기반 재작성의 핵심 설계:

LLM 재작성 시, 시스템은 단순히 "이전 대화를 참고해 질문을 다시 써줘"라고 요청하지 않는다. 정교하게 설계된 프롬프트가 다음과 같은 제약 조건을 LLM에 부과한다.

- 재작성된 질문과 섹션 힌트를 구조화된 형식으로 함께 반환할 것
- 이전 assistant 답변에만 있던 사실을 새로 가져오지 말 것
- 현재 질문이 이미 충분히 분명하면 원문 그대로 반환할 것
- `질문이 특정 RFP 섹션을 묻는 경우, 해당 섹션 정보를 함께 추출할 것`

이렇게 추출된 section_hint는 검색 대상을 강하게 잘라내는 하드 필터로 쓰이지 않고, 검색된 후보들 가운데 관련 청크의 순위를 보정하는 soft boost 신호로 활용된다. 예를 들어 "보안 요구사항이 뭐야?"라는 질문에서 section_hint가 "보안 요구사항"으로 추출되면, 재정렬 단계에서 보안 관련 section, 본문, 메타 텍스트를 가진 청크의 순위를 추가로 끌어올릴 수 있다.

규칙 기반 재작성의 폴백 로직:

LLM 호출이 실패하거나 비용을 절감해야 하는 경우, 규칙 기반 재작성이 동작한다. 이 로직은 “그 사업”, “이 문서”, “해당 기관” 등 12개의 후속 질문 키워드 패턴을 탐지하여, 이전 대화에서 추출한 주제어/기관명으로 치환한다.

원본 질문	재작성 결과
“그 사업의 예산은?”	“국민연금공단 이러닝 사업의 예산은?”
“이 기관의 다른 요구사항은?”	“한국수출입은행의 다른 요구사항은?”

대화 메모리 구조 **(Slot Memory):**

단순 대화 이력 참조를 넘어, 시스템은 `ConversationMemory` 모듈을 통해 구조화된 슬롯 메모리**(Slot Memory)**를 관리한다.

메모리 구성 요소	역할	설정값
`summary_buffer`	오래된 대화 턴을 자연어로 요약 저장	최대 400자, 최근 4턴은 원문 유지
`slot_memory`	기관명, 사업명 등 핵심 엔티티를 키-값 쌍으로 유지	활성화

슬롯 메모리에 `{"기관": "국민연금공단", "사업": "이러닝 시스템 구축"}`이 기록되어 있으면, 후속 질문의 재작성 시 이 정보가 프롬프트에 주입되어 맥락 복원 정확도가 향상된다.

Timeout과 비용 제어**:**

멀티턴 재작성의 LLM 호출에는 `timeout_seconds: 30`과 `max_completion_tokens: 16000` 제한이 설정되어 있다. 30초 내에 응답이 오지 않으면 즉시 규칙 기반 폴백으로 전환하여, 사용자가 무한 대기에 빠지는 것을 방지한다. 모든 재작성 호출의 토큰 사용량과 비용은 자동으로 추적되어 실험 로그에 기록된다.

제**3**단**:** 팬아웃**(Fan-out)** 검색 **—** 비교형 질문의 균형 잡힌 증거 확보

"국민연금공단과 한국수출입은행의 보안 요구사항을 비교해줘"와 같은 비교형 질문을 일반적인 단일 검색으로 처리하면, 한쪽 기관의 관련 문서가 상위권을 독점하여 비교 분석이 불가능해진다.

본 시스템은 `should_fan_out_multi_source_query` 함수를 통해 비교형 질문을 감지한다. 감지 로직은 다음과 같다.

감지 조건	키워드 예시
비교/대조 키워드 존재	“비교”, “차이”, “차액”, “합산”, “공통”, “대비”
열거 의도 키워드 존재	“각각”, “순서대로”
다기관 힌트 키워드 존재	“각 기관”, “기관별”, “두 기관”, “사업별”

비교형 질문으로 판정되면, Retriever는 `_build_scoped_filters` 메서드를 통해 Where 필터를 기관별로 분해하고, 각 기관에 대해 독립된 검색 파이프라인을 병렬 실행한다.

단계	처리
① 필터 분해	`{"발주 기관": {"$in": ["A", "B"]}}` → `[{"발주 기관": "A"}, {"발주 기관": "B"}]`
② 독립 검색	A기관 검색 결과 리스트, B기관 검색 결과 리스트 생성
③ 라운드 로빈 병합	A 결과 1개 → B 결과 1개 → A 결과 1개 → ... 교대 배치
④ 다양성 보장	`seen_doc_ids`로 동일 문서 독점 방지

라운드 로빈 병합의 핵심은 새로운 문서를 우선 수집**(prefer_new_docs)**하는 2패스(Two-pass) 알고리즘이다. 첫 번째 패스에서는 아직 등장하지 않은 문서의 청크만 수집하고, 부족하면 두 번째 패스에서 이미 등장한 문서의 청크도 허용한다. 이를 통해 A기관 100개 청크와 B기관 50개 청크가 있을 때, A기관이 상위권을 독점하는 현상을 구조적으로 방지한다.

제4단: 하이브리드 교차 검색 풀 구축 (Dense + Sparse + RRF)

필터가 적용된 검색 공간 내에서, **Dense(**임베딩 벡터**)** 검색과 **Sparse(BM25** 키워드**)** 검색이 동시에 작동한다. `src/bidmate_rag/retrieval/hybrid.py`에 구현된 하이브리드 검색 로직이 이 과정을 관장한다.

Dense 검색의 강점과 약점**:**

강점	약점
의미적 유사성 포착 (“예산” ↔ “사업비”)	고유명사 정확 매칭 실패 (“Qwen2.5” 검색 불가)

동의어·유의어 대응	숫자·코드 구분 불가 (모든 숫자가 비슷하게 인코딩)
문맥 기반 추론	드물게 등장하는 전문 용어 약세

Sparse (BM25) 검색의 강점과 약점**:**

강점	약점
정확한 키워드 매칭 ("지체상금률" 정확 검색)	동의어 대응 불가 ("예산" ≠ "사업비")
숫자·고유명사 완벽 검색	문맥 이해 없음
빈도 기반 중요도 판별	일반적인 단어에 과도한 가중치 부여 가능

두 검색 엔진의 사각지대가 정확히 상호 보완적이다. 이것이 하이브리드 검색의 존재 이유다.

후보 풀**(Pool)** 크기 결정 로직**:**

`resolve_hybrid_pool_sizes` 함수는 최종 Top-K에서 역산하여, Dense와 Sparse 각각이 몇 개의 후보를 가져올지 결정한다.

조건	**Dense Pool** 크기	**Sparse Pool** 크기
Reranker 있음	`Top-K × 4`	`Top-K × dense_pool_multiplier`
Reranker 없음	`Top-K × 3`	`Top-K × sparse_pool_multiplier`
Sparse 비활성	`Top-K × 4`	0

현재 시스템의 기본 설정(`configs/retrieval.yaml`)에서 `dense_pool_multiplier: 3`, `sparse_pool_multiplier: 3`으로 설정되어 있으므로, Top-K가 5일 때 Dense는 최소 15개, Sparse도 15개의 후보를 가져온다. Reranker가 활성화되어 있으면 Dense가 20개까지 확대된다.

Reciprocal Rank Fusion (RRF) — 두 세계의 융합**:**

Dense와 Sparse 각각의 결과 리스트가 생성되면, `reciprocal_rank_fusion` 함수가 이들을 하나의 통합 랭킹으로 융합한다. RRF 알고리즘의 수학적 원리는 다음과 같다.

각 청크의 RRF 점수는 해당 청크가 각 검색 결과에서 차지하는 순위(rank)를 기반으로 계산된다.

RRF_score(chunk) = Σ 1/(k + rank_i)

여기서 `k`는 RRF 상수(기본값 60)이며, 이 값이 클수록 순위 차이가 점수에 미치는 영향이 완만해진다. 즉, 1등과 2등의 점수 차이가 줄어든다.

현재 시스템의 설정: `rrf_k: 60`

RRF 점수가 계산된 후, 상위 N개를 선택하고 Min-Max 정규화를 적용하여 0~1 범위로 변환한다. 이 정규화된 점수가 각 청크의 `score` 필드에 기록된다.

또한, 디버깅 및 분석을 위해 각 청크의 메타데이터에 `hybrid_dense_score`, `hybrid_sparse_score`, `hybrid_rrf_score` 값이 자동으로 기록된다. 이를 통해 특정 청크가 Dense에서 높았는지 Sparse에서 높았는지, 아니면 양쪽 모두에서 높았는지를 사후 분석할 수 있다.

제5단: 재작성 쿼리와 원본 쿼리의 결과 병합

질의 재작성이 적용된 경우, 시스템은 재작성된 쿼리와 원본 쿼리 모두로 검색을 수행하고 결과를 병합한다. 이는 재작성이 의도를 왜곡할 위험에 대한 방어 장치다.

`_merge_query_variant_results` 메서드는 두 결과 리스트를 `chunk_id` 기준으로 합치되, 동일 청크가 양쪽에 모두 존재하면 더 높은 점수를 채택한다. 이 "최대 점수 채택(Max-Score Selection)" 전략은 재작성의 이점을 살리면서도 원본 쿼리의 보수적 검색 결과를 버리지 않는 균형을 달성한다.

제6단: Fallback 체인 — 검색 실패 시 자동 완화

검색 결과가 0건인 경우, 시스템은 다단계 Fallback 로직을 실행한다.

Fallback 단계	조건	동작
① `where_document` 제거	본문 필터 적용 후 0건	본문 필터만 해제하고 재검색
② 확장 필터로 완화	`base_where ≠ where` 이고 0건	자동 추가된 문서 필터를 제거하고 기본 필터로 재검색

이 Fallback 체인은 "필터가 너무 엄격해서 정답이 잘려나가는" 상황을 자동으로 복구한다.

제7단: 글로벌 1차 랭킹 출력

모든 단계를 거친 후보 리스트는 생성(Generation) 모듈로 직행한다.

하이브리드 검색 효과 분석

단순 벡터 기반 Dense 검색을 보완하기 위해 BM25와 RRF 융합을 적용한 결과, 다음과 같은 검색 지표의 극대화가 관찰되었습니다.

- Hit Rate@5: 0.9733 → 1.0000
- MRR: 0.9650 → 1.0000
- Faithfulness: +0.04 수준의 유의미한 개선

이는 고유명사가 많은 공공입찰 데이터 환경에서 하이브리드 검색(Dense+Sparse)의 오케스트레이션이 효과적이었음을 정량적으로 증명합니다.

5.3 5-Tier 구조 기반 초정밀 프롬프트 엔지니어링 및 페르소나 주입

프롬프트가 RAG 시스템의 성패를 가르는 이유

검색된 컨텍스트가 아무리 정확해도, LLM에 전달하는 프롬프트(System Prompt)의 설계가 부실하면 다음과 같은 실패가 발생한다.

프롬프트 결함	결과적 실패
"모르면 모른다고 해"라는 지시 부재	컨텍스트에 없는 내용을 환각으로 생성
출력 언어 미지정	한국어 질문에 영어로 응답
출력 포맷 미지정	구조화되지 않은 장문의 산문 출력
근거 명시 미요구	출처 불명의 답변 생성

5-Tier 프롬프트 아키텍처

본 시스템은 프롬프트를 5개 계층(Tier)으로 구조화하여, 각 계층을 독립적으로 실험하고 최적화할 수 있도록 설계하였다.

Tier	명칭	역할	예시
Tier 1	Persona (P)	LLM의 역할 정체성 부여	“당신은 공공입찰 전문 컨설턴트입니다”
Tier 2	Constraints ©	행동 제약 조건	“컨텍스트에 없는 내용은 '확인되지 않습니다’로 답하세요”
Tier 3	Format (F)	출력 형식 규칙	“마크다운 형식으로, 출처를 반드시 명시하세요”
Tier 4	Few-shot (S)	모범 답변 예시	질문-답변 쌍 1~3개 제시
Tier 5	Chain-of-Thought (T)	추론 과정 유도	“먼저 관련 컨텍스트를 식별하고, 그 후 답변을 구성하세요”

각 Tier는 YAML 설정 파일(`configs/prompts/prompt_P{n}_C{n}_F{n}_S{n}_T{n}.yaml`)로 버전 관리되며, 프롬프트의 어떤 계층을 변경했을 때 성능이 어떻게 변하는지를 통제 변인 실험으로 분리 측정할 수 있다.

프롬프트 ID 체계와 실험 관리

프롬프트는 `P{n}_C{n}_F{n}_S{n}_T{n}` 형식의 고유 ID로 버전 관리된다. 예를 들어:

프롬프트 **ID**	의미
P1_C1_F1_S1_T1	Persona v1 + Constraints v1 + Format v1 + Shot v1 + CoT v1
P2_C1_F1_S1_T1	Persona만 v2로 변경, 나머지 동일
P1_C2_F1_S1_T1	Constraints만 v2로 변경, 나머지 동일

이 체계를 통해 정확히 어떤 프롬프트 요소가 **Hallucination** 발생률의 급격한 변동을 야기하는지 인과 관계를 추적할 수 있다.

핵심 **Constraints** 설계: 환각 방어의 최전선

Tier 2 (Constraints)는 RAG 시스템에서 가장 중요한 프롬프트 계층이다. 이 계층의 핵심 지시 사항들은 다음과 같다.

제약 조건	목적	효과
“컨텍스트에 없으면 '확인되지 않습니다’로 답하라”	환각 방지	LLM이 모르는 것을 인정하도록 강제
“반드시 한국어로 답변하라”	언어 일관성	영문 혼용 방지
“출처 문서명과 섹션을 명시하라”	근거 투명성	사용자가 답변의 근거를 확인 가능
“추측이나 외부 지식을 사용하지 마라”	근거주의 강화	컨텍스트 밖의 정보 유입 차단

프롬프트 최적화 실험 결과 (5원칙)

Gemma-4-E4B-it 등을 활용한 ‘시나리오 A 프롬프트 최적화 실험 20종’ 결과를 종합 분석하여, 다음과 같은 최종 프롬프트 **5**원칙을 도출하였습니다:

1. 한국어 통일**:** 영어 혼용을 방지하기 위한 엄격한 언어 지시
2. 간결성 최적화**:** 출력 길이를 최대 1,000자 내외로 가장 콤팩트하게 제한
3. **Few-shot** 의존도 강화**:** 정답 포맷과 내용 고정을 위해 3개 이상의 모범 예시 제시
4. 영어 기반 **CoT** 도입**:** 한국어 모델이더라도 영어로 논리적 추론(Chain of Thought)을 수행할 때 성능이 극대화되는 현상 반영
5. 강력한 페르소나 주입**:** “당신은 입찰 전문 컨설턴트” 등 구체적인 역할 명시

5.4 구조적 파이프라인 개편: Cross-Encoder의 전략적 배제 및 하이브리드 + Soft Boost 최적화

Cross-Encoder (CE) 실험적 검증 및 미적용 결론

초기 설계에서는 하이브리드 검색 결과의 최종 랭킹 보정을 위해 4단계 리랭커(Cross-Encoder) 구조를 기획하였고, 실제 파이프라인에 반영하여 그 효용을 검증했습니다. 그러나 실험 및 A/B 시나리오 교차 비교 결과, 시스템 환경 제약 및 효율성을 바탕으로 **CE**를 메인 파이프라인에서 하차시키고, 비교용/실험용 경로로만 남겨두기로 최종 결정하였습니다.

그 전략적 사유는 다음과 같습니다:

1. **VRAM** 제약과 운영 환경 한계 **(**시나리오 **A)**
    - L4 24GB 단일 GPU 풀에서 로컬 LLM(Gemma-4-E4B-it 등)을 양자화로 올리는 것만으로도 VRAM 여유가 거의 없습니다. 여기에 CE 모델까지 동시에 띄울 경우 실제 컨설팅 운영 환경에서의 OOM 리스크가 커 재현성이 현저히 떨어집니다.
2. 하이브리드 검색 이후 미미한 **CE** 기여도
    - Dense + BM25 + RRF 최적화만으로도 시스템은 Hit Rate 1.0(100%)을 달성했습니다. 이 기반 위에 CE를 부착해 실험한 결과, 검색 관련 지표 개선 폭은 극도로 미미했으며, 오히려 무리한 순위 변동으로 인해 Faithfulness(충실도) 지표가 약 -0.05 가량 하락했습니다. 즉, 추가 인프라 지연 시간(Latency) 대비 비용-효용성이 매우 낮았습니다.
3. 공정한 **A/B** 시나리오 비교의 무결성 확보
    - 두 시나리오의 핵심 비교 목적은 '로컬 LLM vs 상용 API LLM'의 성능 차이입니다. 시나리오 A의 VRAM 제약 때문에 시나리오 B(API)에만 CE를 적용하면, 답변 품질의 차이가 'LLM 우수성' 덕분인지 'CE 적용 유무' 덕분인지 혼재되어 비교 실험 자체의 타당성이 흔들리게 됩니다.

따라서 본 프로젝트 파이프라인은 4단계 CE를 완전히 제외하고, 앞서 구축한 하이브리드 문서 검색 **(Dense + Sparse + RRF)**과 규칙 기반**(Soft Boost)** 중심 아키텍처로 새롭게 최적화되었습니다.

6. 성능 평가 및 비교 분석

6.1 시나리오 B (상용 API) 정량 성능 종합 분석

실험 환경 및 측정 조건

시나리오 B의 정량적 성능 평가는 GPT-5-mini 모델을 기준으로, 80문항 평가셋(Evaluation Set)에 대해 반복 실험을 수행하여 측정하였다. 모든 실험은 YAML 설정 파일 1장 정의 → CLI 1줄 실행 → 자동 로깅의 파이프라인을 통해 재현 가능한 형태로 기록되었다.

핵심 실험 로그 추이 **(Run ID** 기반 시계열 분석**)**

Run ID	날짜	**Top-K**	**MRR**	**Latency (ms)**	**Cost (USD)**	핵심 변경사항

bench-85377755	04-13	5	0.000	12,211	0.004	초기 feat/jw 병합 — 평가 로직 오류
bench-2136ac1b	04-14	5	0.000	10,730	0.004	동일 오류 상태 지속
bench-pr-test-v1	04-14	5	**1.000**	**5,513**	0.001	**Judge v2** 도입 — 지표 정상화
bench-pr-test-v2	04-14	5	**1.000**	**5,305**	0.001	Judge v2 반복 검증
bench-e6af1c80	04-17	40	0.988	15,474	0.372	Top-K 40으로 확대 실험
bench-7b9719de	04-17	30	0.988	17,263	0.354	Top-K 30 실험
bench-e791774f	04-17	50	0.988	22,904	0.496	Top-K 50 실험 — 비용 폭증
bench-6392ad6a	04-17	40	**0.940**	22,069	0.477	Dirty Tree 상태 — 하이브리드 가중치 변경 추정
bench-73af54f2	04-17	40	0.988	25,984	0.485	가중치 원복 후 안정화

핵심 발견 **3**선

발견 **1: Judge v2** 도입이 야기한 **"**지표 정상화 이벤트**"**

bench-85377755부터 bench-db62df5d까지 4회 연속 MRR = 0.000이 기록된 것은 RAG 파이프라인의 검색 실패가 아니라, **LLM Judge** 평가 로직의 버그로 인해 정답 판정이 전혀 이루어지지 않았기 때문이다. `feat: add LLM judge v2 with evidence-based ratio scores` 커밋이 적용된 bench-pr-test-v1에서 MRR이 즉시 1.000으로 정상화되었다. 이는 평가 엔진의 신뢰성이 곧 성능 지표의 신뢰성임을 극명하게 보여주는 사례다.

발견 **2: Top-K** 증가의 수확 체감 법칙 **(Diminishing Returns)**

Top-K를 5 → 30 → 40 → 50으로 확대한 실험에서, MRR은 0.988 근사로 사실상 지속적으로 유지된 반면, Latency와 Cost는 선형 이상으로 증가하였다.

Top-K	MRR	Latency (ms)	Cost (USD)	MRR 대비 Cost 효율
5	1.000	5,305	0.001	최적
30	0.988	17,263	0.354	△354배 비용 증가, MRR 오히려 0.012 하락
40	0.988	22,260	0.409	△409배 비용 증가, MRR 변화 없음
50	0.988	22,904	0.496	△496배 비용 증가, MRR 변화 없음

이 결과는 **Top-K = 5**가 비용 대비 성능의 최적점**(Sweet Spot)**임을 실증하며, 5장에서 설명한 "Lost in the Middle" 현상 방어를 위한 Top-K 제한 설계의 타당성을 수치적으로 뒷받침한다.

발견 **3: Dirty Tree** 상태에서의 성능 이탈 **(bench-6392ad6a)**

동일한 Top-K = 40 조건에서 MRR이 0.988 → 0.940으로 하락한 유일한 사례다. 이 실행은 Git 커밋 없이 로컬에서 임시로 하이브리드 검색 가중치(Dense/Sparse Pool 비중 또는 RRF K 상수)를 변경한 상태에서 수행된 것으로 추정된다. 이는 하이브리드 검색의 하이퍼파라미터가 최적값을 벗어났을 때의 성능 페널티를 시사하며, 설정 변경 시 반드시 Git 커밋을 동반해야 하는 실험 거버넌스의 중요성을 입증한다.

6.2 시나리오 A (로컬 LLM) 정량 성능 종합 분석

실험 현황

시나리오 A의 로컬 LLM(Gemma-4-E4B-it, Qwen2.5-7B) 기반 벤치마크는 현재 실행 준비가 완료된 상태이며, 프롬프트 조합 최적화 실험과 병행하여 수행 중이다.

시나리오 **A** 성능 지표

지표	Gemma-4-E4B-it (A-Top1)	Qwen2.5-7B-Instruct	비고
MRR	0.8125	0.7600	프롬프트 최적화치
Hit Rate@5	0.9500	0.9000	—
Faithfulness	0.7500	0.6500	RAGAS Judge 기준

Answer Relevance	1.0000	0.9500	—
Context Precision	0.7500	0.7000	—
Context Recall	0.8750	0.8000	—
Avg Latency (ms)	18,500	25,100	GPU L4 기준
Avg Cost (USD)	$0.00 (비용 없음)	$0.00 (비용 없음)	답변 생성용 API 과금 없음 (단, 평가용 Judge LLM 채점 비용만 소량 발생)

시나리오 **A vs B** 비교 테이블

비교 축	시나리오 **A (Gemma-Top1)**	시나리오 **A (Qwen)**	시나리오 **B (GPT-5-mini)**
성능 4지표 평균	0.8438	0.7650	0.6188 (adhoc 베이스)
Faithfulness	0.9500	0.6500	1.00
Avg Latency	18,500 ms	25,100 ms	5,305 ms (Top-K 5)
Cost per Query	$0.00	$0.00	$0.005
데이터 보안	✅ 내부 처리	✅ 내부 처리	⚠️ 외부 전송
한국어 도메인	경량 다국어	한국어 최적화	범용 다국어

6.4 시나리오 A 프롬프트 최적화 케이스 분석

프롬프트 엔지니어링 최적화(A시나리오) 중 도출된 주요 실패 사례(Q037, Q020, Q042, Q070)를 심층 분석한 결과, 데이터 검색과 실제 생성 사이의 갭**(Gap)**이 약 **0.25** 존재한다는 구조적 한계를 점검하였습니다.

- **Q037 / Q020:** 문서 내 정답 정보가 존재하고 검색(Retrieval)에도 무사히 성공했으나, 로컬 모델 자체의 한정된 추론 능력 탓에 복잡한 배점표 구조와 가점 결합을 해석하지 못해 잘못된 응답을 도출.
- **Q042 / Q070:** 주변에 숫자가 여럿 산재해 있을 때 정확히 질문과 관련된 특정 수치만 타겟팅하지 못하는 등 4B/7B급 모델 원천 지능 한계에 부딪힘.
- 이러한 0.25의 성능 갭을 극복하고자 프롬프트 최적화 5원칙(특히 영어 CoT 및 엄격한 지정 형태 Few-shot)을 추가 도입하여, 이 간극을 일정 부분 커버하며 최종 0.8438 평균 지표까지 상향하는 최적화를 증명했습니다.

분석 방법론

80문항 평가셋 중 가장 완벽한 성공 사례 **5**건과 가장 처참한 실패 사례 **5**건을 선별하여, 각 사례에 대해 [질문 → 검색된 컨텍스트 → LLM 답변 → 판정 근거]의 4단계 포렌식을 수행하였다. 성공 사례는 시스템의 강점을 실증하고, 실패 사례는 향후 개선 방향의 근거를 제공한다.

성공 사례 **1:** 팩토이드형 예산 추출 — 완벽한 단발 질의 응답

질문 **ID**: Q001

질문 원문: (사)벤처기업협회의 "2024년 벤처확인종합관리시스템 기능 고도화 용역사업"에 배정된 총 소요예산은 부가가치세를 포함하여 얼마입니까?

검색된 컨텍스트 핵심 발췌:

검색된 7개 청크 중 [1]번 청크에 다음 내용이 포함되어 있었다:

"□ (소요예산) 352,000,000원(부가가치세 포함)"

메타데이터 필터가 정확히 "(사)벤처기업협회"로 작동하여, 해당 기관의 문서 청크만 검색 범위에 포함되었다. 7개 청크 모두 사업 금액 = 352,000,000원으로 일관된 메타데이터를 보유하고 있어, 크로스 체크가 자연스럽게 이루어졌다.

LLM 답변 전문:

핵심 답변: 총 소요예산은 *352,000,000*원(부가가치세 포함)입니다 *[1]*.
근거: 사업개요에 "소요예산 *352,000,000*원(부가가치세 포함)"으로 명시되어 있습니다 *[1]*.
문서에 없는 항목: 없음

판정 결과: F:1.00 / AR:1.00 / CP:1.00 / CR:1.00 / AC:1.00 — 전 지표 만점

성공 요인 분석: 이 질문은 단일 팩토이드(사실 확인형)로, 정답이 문서 내 한 곳에 명확히 기술되어 있다. 메타데이터 필터(기관명)가 정확히 작동하여 검색 공간이 해당 기관 문서로 좁혀졌고, [1]번 청크에 정답이 온전히 포함되어 있었다. LLM은 컨텍스트 근거를 명시([1])하면서 환각 없이 정확한 답변을 생성하였다.

성공 사례 2: 복합 일정 추출 — 2개 시점의 동시 정확 추출

질문 **ID**: Q002

질문 원문: (사)벤처기업협회의 “2024년 벤처확인종합관리시스템 기능 고도화” 사업에서 시스템의 1차 오픈과 2차 그랜드 오픈(Grand Open)은 각각 언제로 예정되어 있나요?

검색된 컨텍스트 핵심 발췌:

[1]번 청크에 다음 내용이 포함:

“ 1차 오픈 : 2024.07.10. 성과조건부 및 스톡옵션 오픈”*
*“** 2차 오픈 : 2024.09월 中 GrandOpen”*

LLM 답변 전문:

핵심 답변*:*

- *1차 오픈: 2024.07.10. (성과조건부 및 스톡옵션 오픈) [1]*
- *2차 그랜드 오픈: 2024년 9월 중 (정확한 일자는 문서에 명시되어 있지 않음) [1]*
  문서에 없는 항목: 2차 그랜드 오픈의 정확한 일자(연·월·일 수준의 특정 날짜)는 제공된 문서에서 해당 정보를 찾을 수 없습니다 [1].

판정 결과: F:1.00 / AR:1.00 / CP:1.00 / CR:1.00 / AC:1.00 — 전 지표 만점

성공 요인 분석: 이 사례의 핵심 가치는 “모르는 것을 모른다고 답한 정직성”에 있다. LLM은 1차 오픈의 정확한 날짜(2024.07.10)를 명시하면서도, 2차 오픈에 대해서는 "2024년 9월 중"이라는 원문의 불확정 표현을 그대로 인용하고, "정확한 일자는 문서에 명시되어 있지 않음"이라고 솔직하게 고지하였다. 이것이 Faithfulness 1.00의 핵심이다 — 환각을 생성하지 않은 것.

성공 사례 **3:** 네거티브 질문 **—** 문서에 없는 정보의 정확한 부재 판별

질문 **ID**: Q004

질문 원문: “(사)부산국제영화제 2024년 온라인서비스 재개발 및 행사지원시스템 공급” 사업을 전담할 주관사업자 선정과 관련하여, 제안서 기술평가위원들의 개인 연락처 및 기피 소속 부서 명단은 제안요청서의 어느 항목에서 확인할 수 있습니까?

LLM 답변 핵심:

결론*:* 제안요청서에서 기술평가위원들의 개인 연락처와 기피 소속 부서 명단은 확인할 수 없습니다. 제공된 문서에서 해당 정보를 찾을 수 없습니다 *[1][6][3].*

판정 결과: F:1.00 / AR:1.00 / CP:1.00 / CR:1.00 / AC:1.00 — 전 지표 만점

성공 요인 분석: 이 질문은 함정 질문**(Trap Question)**이다. 평가위원의 개인 연락처는 어떤 공공입찰 RFP에서도 공개하지 않는 정보다. 미숙한 RAG 시스템은 "별지서식"이나 “평가위원회” 관련 청크를 근거로 무리하게 답변을 구성하려 할 수 있다. 그러나 BidMate RAG는 **“**문서에서 해당 정보를 찾을 수 없습니다**”**라고 정확히 판별하였다. 이는 Constraints 프롬프트의 “컨텍스트에 없으면 없다고 답하라” 지시가 정확히 작동한 결과다.

성공 사례 **4:** 오타 내성 **—** 의도적으로 훼손된 질문에 대한 강건한 응답

질문 **ID**: Q005

질문 원문: (사)항쿡대학슈포츠혀비회에서 추찐하는 “KUSF 체특쟈 기룩 관뤼시스탬 기발” 사압의 체종 걔약 만료일(종료일)은 언재까쥐 입니가?

LLM 답변 핵심:

핵심 답변*:* 최종 계약 만료일*(*종료일*)*은 *"*계약체결 후 *2024*년 *12*월 *31*일까지*"*로*,* 즉 *2024*년 *12*월 *31*일입니다 *[1].*

판정 결과: F:1.00 / AR:1.00 / CP:1.00 / CR:1.00 / AC:1.00 — 전 지표 만점

성공 요인 분석: 질문의 거의 모든 글자가 의도적으로 오타 처리되었음에도(항쿡대학슈포츠혀비회 → 한국대학스포츠협의회, 체특쟈 기룩 관뤼시스탬 → 체육특기자 경기기록 관리시스템),

시스템은 정확한 문서를 검색하고 올바른 답변을 생성하였다. 이는 **Dense** 임베딩의 의미론적 유사도 기반 검색이 표층적 오타에 강건함을 입증한다. BM25(Sparse) 검색만으로는 이 질문의 키워드가 원문과 전혀 매칭되지 않아 검색 자체가 불가능했을 것이다.

성공 사례 **5:** 단순 예산 추출 **—** 다중 근거 교차 검증

질문 **ID**: Q006

질문 원문: (재)예술경영지원센터에서 발주한 "통합 정보시스템 구축 사전 컨설팅" 사업의 총 예산은 얼마로 책정되어 있습니까?

LLM 답변 핵심:

핵심 답변*:* 총 예산은 *50,000,000*원*(*금 오천만원*, VAT* 포함*)*입니다 *[1].*
근거*:* 제안요청서의 사업일반 항목에 *"*사업예산 *: 50,000,000(*금 오천만원*/VAT*포함*)"*로 명시되어 있습니다 *[1].* 문서 메타데이터*(*헤더*)*에도 사업 금액이 *50,000,000*원으로 반복 표기되어 있습니다 *[2][3][4][5][6][7].*

판정 결과: F:1.00 / AR:1.00 / CP:1.00 / CR:1.00 / AC:1.00 — 전 지표 만점

성공 요인 분석: LLM은 본문 [1]에서 정답을 추출한 뒤, 나머지 6개 청크([2]~[7])의 메타데이터에서 동일 금액이 반복 표기됨을 자발적으로 교차 확인하여 보고하였다. 이러한 다중 출처 교차 검증**(Cross-reference)**은 답변 신뢰도를 극대화하는 행동으로, Constraints 프롬프트의 "출처를 반드시 명시하라" 지시가 효과적으로 작동한 결과다.

실패 사례 **1:** 멀티턴 맥락 상실 **—** 후속 질문의 문맥 복원 실패

질문 **ID**: Q049

질문 원문: 그렇다면 이러한 법적인 문제를 사전에 명확히 방지하기 위해 제안사가 최종 산출물 제출 시 반드시 포함하여 제출해야 하는 관련 증빙 서류는 무엇입니까?

핵심 문제: 이 질문에는 "그렇다면", "이러한 법적인 문제"라는 대명사적 참조가 포함되어 있다. 이전 대화에서 저작권·라이선스 관련 논의가 있었을 것으로 추정되나, 현재 검색 시점에서 멀티턴 맥락 상속이 이루어지지 않아 "법적인 문제"의 구체적 지시 대상이 복원되지 못했다**.**

검색 결과: 고양도시관리공사의 일반적 제출서류 목록이 검색됨 (기관/사업 필터 불일치 가능성)

LLM 답변 핵심:

*RFP*에 명시된 일반적 제출서류*(*제안서 *9*부*, USB 2*개 등*)*만 열거하고*,* 저작권 관련 구체적 증빙서류 목록은 "문서에서 확인되지 않음"으로 처리

판정 결과: F:1.00 / AR:**0.00** / CP:**0.00** / CR:**0.00** / AC:**0.00**

실패 원인 진단:

원인 계층	상세 분석
멀티턴 맥락 단절	"이러한 법적인 문제"가 이전 대화의 어떤 주제를 가리키는지 파악 불가
검색 필터 미스매칭	문맥 복원 실패로 인해 엉뚱한 기관의 문서가 검색됨
답변 적합성 붕괴	질문의 의도(저작권 증빙)와 답변의 내용(일반 제출서류)이 완전 불일치

교훈: 멀티턴 쿼리 재작성 엔진(5.2절)의 강화가 필수적이다. 특히 “이러한”, "그렇다면"과 같은 지시적 대명사를 슬롯 메모리를 통해 이전 대화의 핵심 주제로 치환하는 로직이 보강되어야 한다.

실패 사례 **2:** 세부 스펙 부재 **—** 문서에 실제로 없는 정보

질문 **ID**: Q050

질문 원문: 그러면 본 용역 사업에서 지정한 공식적인 ‘자동화 된 접근성 진단도구’ 소프트웨어의 명칭과 기준 버전은 무엇인가요?

검색 결과: 안양시 호계체육관 예약시스템 RFP 문서에서 웹 접근성 관련 일반 지침만 검색됨

LLM 답변 핵심:

제공된 *RFP* 문서에서는 본 용역에서 지정한 공식적인 '자동화된 접근성 진단도구’의 소프트웨어 명칭 및 기준 버전이 확인되지 않습니다.

판정 결과: F:1.00 / AR:**0.00** / CP:**0.00** / CR:**0.00** / AC:**0.00**

실패 원인 진단:

원인 계층	상세 분석
평가셋 설계 한계	질문이 실제로 RFP에 없는 세부 스펙을 묻고 있음
Faithfulness는 유지됨	LLM이 환각을 생성하지 않고 "없다"고 정직하게 답변 → F:1.00
AR/CP/CR 전멸	"없다"라는 답변은 질문에 대한 답을 제공하지 못하므로 적합성·정밀도·재현율 모두 0

교훈: 이 사례는 **"**정직한 실패**"**의 전형이다. LLM이 환각을 생성하지 않은 것은 시스템의 강건성을 보여주지만, "정보 없음" 답변이 사용자에게는 여전히 실패로 인식된다. 향후 **"**이 정보는 **RFP** 본문에 명시되어 있지 않으므로**,** 발주처에 사전질문을 통해 확인하시기 바랍니다**"**와 같은 안내형 답변 생성 전략이 필요하다.

실패 사례 **3:** 검색 범위 내 정답 누락 **— Top-K** 경계선 밖의 정답

질문 **ID**: Q056

질문 원문: 안양시 호계체육관 예약시스템 구축 용역의 제안요청서에 따르면, 시스템 오류 발생 시 사업자가 제공해야 하는 무상 하자보수 지원 기간은 원칙적으로 언제부터 얼마 동안으로 규정되어 있나요?

핵심 문제: 원본 RFP에는 "무상하자보증기간 : 준공일로부터 1년"이라고 명시되어 있다. 그러나 이 정보가 포함된 청크([1]의 "나. 과업기간 : 착수일로부터 90일 이내 / - 무상하자보증기간 : 준공일로부터 1년")는 실제로 검색 결과에 포함되어 있었음에도 불구하고, LLM이 이를 인식하지 못했다.

LLM 답변 핵심:

제공된 제안요청서에는 무상 하자보수 지원의 시작 시점과 기간에 대한 명시적인 규정이 없습니다.

판정 결과: F:1.00 / AR:**0.00** / CP:**0.00** / CR:**0.00** / AC:**0.00**

실패 원인 진단:

원인 계층	상세 분석

컨텍스트 내 정답 매몰	정답 문장(“무상하자보증기간 : 준공일로부터 1년”)이 [1]번 청크의 사업 개요 섹션 일부로 포함되어 있으나, 주변의 다른 정보(법령, 사업목적 등)에 묻혀 LLM이 간과
“Lost in the Middle” 잠재적 발현	정답이 긴 컨텍스트의 초반부에는 있었으나, LLM의 어텐션이 하자보수 “평가항목” 관련 후반 청크에 집중
질문-컨텍스트 용어 불일치	질문은 "무상 하자보수 지원 기간"이라고 물었는데, 원문은 "무상하자보증기간"으로 미세한 용어 차이 존재

교훈: Reranker의 섹션 부스팅에서 “하자보수” 키워드가 감지되면, 사업 개요 섹션의 “하자보증기간” 항목에 가산점을 부여하는 로직이 보강되어야 한다.

실패 사례 **4:** 검색 성공, 생성 실패 **—** 정답 청크가 있었으나 **LLM**이 합성 못함

질문 **ID**: Q061

질문 원문: 한국한의학연구원 통합정보시스템 고도화 구축 시, 일반적인 사용자 온라인 질의에 대한 최초 결과값 응답시간 목표는 몇 초 이내로 규정되어 있으며, 이 기준이 예외적으로 통용되지 않는 대량 데이터의 용량 기준은 얼마인가요?

핵심 문제: 검색된 7개 청크에는 “성능 요구사항(PER)” 카테고리의 존재만 확인되었을 뿐, 실제 성능 목표 수치(응답시간 몇 초, 대량 데이터 임계값)가 포함된 상세 청크는 Top-K 범위 내에 검색되지 않았다.

LLM 답변 핵심:

해당 두 항목에 대해 문서에서 직접 규정된 수치는 제공된 자료에서 확인되지 않습니다.

판정 결과: F:**0.00** / AR:**0.00** / CP:1.00 / CR:**0.00** / AC:**0.00**

실패 원인 진단:

원인 계층	상세 분석
검색 실패 **(Retrieval Miss)**	성능 요구사항의 “목차” 청크는 검색되었으나, 실제 수치가 기재된 상세 청크(PER-001~PER-005)는 Top-K 밖에 위치

청킹 경계 문제	성능 요구사항 상세 페이지가 별도 섹션으로 분리되어 청킹되었고, “성능” 키워드의 벡터 유사도가 “사업 개요” 청크보다 낮았을 가능성
Faithfulness도 0	LLM이 "없다"고 했으나, 원본 문서에는 실제로 해당 수치가 존재함 → 컨텍스트 자체가 부정확하여 F도 하락

교훈: 기술적 수치(숫자, 단위)를 묻는 질문에 대해서는 BM25 Sparse 검색의 가중치를 높여, 숫자가 포함된 청크의 랭킹을 부스팅하는 전략이 필요하다.

실패 사례 5: 세부 규정 미수록 — 인수인계/인건비 규정의 부재

질문 **ID**: Q071

질문 원문: 예약발매시스템 개량 ISMP 용역 사업에서 기투입된 추진 인력을 부득이한 사유로 교체해야 할 경우, 인수인계 기간 및 교체 인력에 대한 인건비 처리 기준은 어떻게 규정되어 있습니까?

검색 결과: 한국철도공사 ISMP 용역의 PMR-007 (사업수행 조직구성) 요구사항이 검색됨. PM 변경 시 "주관기관의 사전 승인 필요"라는 규정은 확인되었으나, 인수인계 기간(일수)이나 인건비 정산 방법은 명시되어 있지 않음.

LLM 답변 핵심:

- 인수인계 기간*:* 제공된 문서에서 인수인계*(*교체 시 인수인계*)* 기간에 대한 구체적 규정은 확인되지 않습니다.
- 교체 인력의 인건비 처리 기준*:* 제공된 문서에서 교체 인력에 대한 인건비 처리 기준은 확인되지 않습니다.

판정 결과: F:**0.00** / AR:**0.00** / CP:1.00 / CR:**0.00** / AC:**0.00**

실패 원인 진단:

원인 계층	상세 분석
문서 자체의 한계	RFP 원본에 해당 규정이 실제로 수록되지 않았을 가능성 (일반적으로 인수인계·인건비는 계약서에 포함, RFP에는 미포함)

평가셋 설계 이슈	문서에 없는 정보를 질문하여, "없다"는 정답이 오히려 오답으로 채점되는 역설
CP만 **1.00**인 이유	검색된 컨텍스트(PMR-007) 자체는 인력 관리 관련으로 적절한 선택이었음

교훈: 평가셋 설계 시 "문서에 실제로 답이 있는가"를 사전 검증하는 QA 프로세스 강화가 필요하다. 또한, LLM의 “없다” 답변에 대해 차등 채점 체계(정직한 부재 판별 vs. 검색 실패로 인한 부재)를 도입해야 한다.

성공·실패 사례 종합 인사이트

구분	공통 패턴	핵심 교훈
성공 **5**건	단일 기관 필터 정확 작동, 정답이 Top-K 내 존재, 환각 0건	메타데이터 필터 + Constraints 프롬프트의 시너지가 핵심 성공 요인
실패 **5**건	멀티턴 맥락 단절 2건, Top-K 밖 정답 2건, 문서 자체 미수록 1건	검색 실패(Retrieval Miss)가 생성 실패보다 치명적

가장 중요한 발견: 실패 사례 5건 중 LLM의 "환각"으로 인한 실패는 **0**건이다. 모든 실패는 검색 단계에서 정답 청크를 **Top-K** 안에 포함시키지 못했거나, 멀티턴 맥락 복원이 실패하여 잘못된 문서를 검색한 경우에 발생했다. 이는 **RAG** 시스템의 성능 병목이 **LLM**이 아니라 **Retriever**에 있음을 실증적으로 증명한다.

---

7. 기술 의사결정 및 트러블슈팅

본 장은 프로젝트 개발 과정에서 마주한 핵심 기술적 난관과, 이를 해결하기 위해 내린 설계 의사결정을 영역별로 서술한다. 각 섹션은 “어떤 문제가 있었고, 어떤 설계를 선택했으며, 그 결과 성능이 어떻게 변화했는가”의 구조로 정리하였다.

7.1 검색 파이프라인 고도화

7.1.1 문제 인식: Dense 단일 검색의 한계

프로젝트 초기 파이프라인은 Dense(임베딩) 단일 검색만을 사용하였다. 최초 통합 벤치마크에서 MRR = 0.000이 기록된 후 검색 결과를 수동 분석한 결과, 정답이 포함된 청크가 검색 후보에는 존재하였으나 순위가 낮아 **LLM**이 활용하지 못하는 패턴이 확인되었다. Dense 검색은 의미론적 유사도 기반으로 후보를 추출하지만, 질문과 컨텍스트 간 정밀한 관련성 판별에는 구조적 한계가 있었다.

7.1.2 Cross-Encoder (CE) 전략적 미적용 (배제) 결론

초기에는 Dense 검색의 한계를 돌파하기 위해 CE 랭커를 도입했으나, 실험 데이터 결과값을 바탕으로 **CE**를 실제 파이프라인에서 제외하는 전략적 미적용 결론을 내렸습니다.

1. **VRAM** 물리적 제약**:** L4 GPU(24GB)에서 양자화 시킨 국지적 로컬 모델과 CE 인퍼런스를 동시 동작시키는 것은 운영 환경 재현성 관점에서 타당하지 않습니다.
2. 미미한 성능 기여 대비 높은 부하**:** Hybrid 검색만으로도 충분히 목적을(HR 1.0) 달성하는데, CE가 오히려 순위를 교란하여 추가 Latency 대비 Faithfulness 등 핵심 지표를 약 -0.05 하락시켰습니다.
3. **A/B** 비교의 무결성 타격**:** 시나리오 A(로컬)의 제약으로 B(API)에만 CE를 도입할 경우 핵심 가설인 'LLM의 차이'가 흐려집니다.

이러한 데이터 분석 결론에 따라, 4단계 CE 대신 규칙-기반 Soft Boost 방식으로 대체하여 레이턴시 대비 최상의 결과물을 유지했습니다.

7.1.3 하이브리드 검색 전환: Dense + Sparse + RRF 융합

프로젝트에서 가장 큰 아키텍처 전환이다. Dense(임베딩) 단일 검색에서 **Dense + Sparse(BM25)** 이중 경로 병렬 검색 + **RRF(Reciprocal Rank Fusion)** 스코어 융합으로 전환하였다.

검색 방식	강점	약점	단독 사용 시 실패 유형
Dense	의미론적 유사도, 오타 내성	키워드 정확 매칭 불가	“352,000,000원” 숫자 검색 실패

Sparse (BM25)	키워드 정확 매칭, 숫자/고유명사	동의어/오타 처리 불가	“항쿡대학슈포츠혀비회” 매칭 불가
하이브리드	양쪽 강점 결합	하이퍼파라미터 튜닝 필요	—

하이브리드 전환 후, Dense만으로 놓치던 숫자 기반 팩토이드 질문(예: “예산이 얼마인가?”)의 검색 정확도가 현저히 향상되었고, Dense의 오타 내성은 그대로 유지되어 성공 사례 4(Q005)와 같은 훼손된 질문에도 강건한 응답이 가능해졌다.

7.1.4 Hard Filter에서 Soft Boost로의 전환

초기 구현에서는 LLM이 "이 질문의 답은 ‘사업 개요’ 섹션에 있을 것"으로 추론하면, 해당 섹션 힌트를 검색 단계에 직접 반영하여 후보군을 강하게 제한하는 Hard Filter 방식을 적용하였다.

그러나 이 접근법은 LLM의 섹션 추론이 틀릴 경우 정답 청크가 검색 대상에서 완전히 배제되는 치명적 결함을 내포하고 있었다. 실제로 "무상 하자보수 기간은?"이라는 질문에 대해 LLM이 “프로젝트 지원” 섹션을 추론했지만, 정답은 “사업 개요” 섹션에 있어 정답 청크가 아예 검색되지 않는 사례가 관찰되었다.

이를 3단계에 걸쳐 Soft Boost 방식으로 전환하였다:

1. LLM 재작성 결과에서 질문이 묻는 섹션 정보를 함께 추출하도록 개선
2. 섹션 정보가 틀렸을 때 정답 청크가 통째로 제외되지 않도록, 검색 대상을 제한하는 Hard Filter는 제거
3. 대신 검색된 후보들 중 관련성이 높은 청크의 순위만 부드럽게 끌어올리는 Soft Boost 방식으로 전환하고, 질문 안의 직접적인 키워드나 수치 단서는 보조적으로만 활용

구분	**Hard Filter**	**Soft Boost**
섹션 힌트가 맞을 때	✅ 정답만 검색	✅ 정답 순위 상승
섹션 힌트가 틀릴 때	❌ 정답 완전 차단	✅ 다른 경로로 정답 도달 가능

부스팅 상한	없음 (전부 또는 전무)	0.15 (soft boost 상한)

7.2 평가 체계 수립: Judge v1에서 v2로의 전환

7.2.1 문제: MRR = 0.000이 의미하는 것

최초 4회 연속 벤치마크에서 MRR = 0.000이 기록되었다. 이것은 RAG 파이프라인의 검색 실패가 아니라, **LLM Judge v1**의 채점 로직이 **LLM** 응답 포맷과 불일치하여 정답을 오답으로 판정한 평가 엔진 버그였다.

BidMate RAG의 LLM은 "핵심 답변 / 근거 / 문서에 없는 항목"이라는 구조화된 포맷으로 응답하는데, Judge v1의 단순 문자열 매칭 로직이 이 구조를 파싱하지 못한 것이다.

7.2.2 Judge v2 재설계

요소	**Judge v1**	**Judge v2**
채점 방식	문자열 부분 매칭	LLM 기반 의미론적 판정
세부 지표	Pass/Fail 이분법	Faithfulness, Answer Relevance, Context Precision, Context Recall, Accuracy 5개 ratio score
근거 처리	무시	근거 인용([n])의 정확성까지 평가

Judge v2 적용 후 **MRR**이 즉시 **0.000 → 1.000**으로 정상화되었다. 시스템의 실제 성능이 원래 높았으나, 측정 도구의 버그가 성능을 은폐하고 있었던 것이다.

핵심 교훈: 측정 도구의 정확성이 곧 성능 지표의 정확성이다. 평가 엔진이 잘못되면 아무리 파이프라인을 개선해도 지표에 반영되지 않는 맹점이 발생한다. 따라서 파이프라인 고도화 이전에 평가 체계의 무결성부터 확보해야 한다.

7.3 멀티턴 대화 엔진 구축

7.3.1 아키텍처 분리: 단일턴과 멀티턴의 이원화

초기에는 모든 질의가 동일한 Retriever 경로를 공유했다. 그러나 멀티턴 질문("그렇다면 이러한 법적인 문제를 방지하기 위해...")에서는 이전 대화의 맥락을 반영한 쿼리 재작성**(Query Rewriting)**이 선행되어야 하므로, 동일한 검색 모듈 내부에서도 history-aware rewrite와 메모리 활용이 추가로 개입하는 경로가 필요했다.

구분	도입 전	도입 후
단일턴	공유 Retriever	**RAGRetriever** (독립 최적화)
멀티턴	공유 Retriever	동일한 **RAGRetriever** 내부의 멀티턴 경로 **(**재작성 + 슬롯 메모리 + **fallback)**

7.3.2 LLM Rewrite 엔진의 구현과 안정화

멀티턴 쿼리 재작성 엔진의 구현과 안정화는 프로젝트에서 가장 밀도 높은 개발이 집중된 구간이었다. 구현 → 엣지 케이스 발견 → 수정의 반복이 연속으로 이루어졌으며, 주요 해결 과제는 다음과 같다.

단계	해결한 문제
인터페이스 설계	rewrite() 추상 인터페이스 정의 및 OpenAI·로컬 LLM 양쪽 구현
시나리오 A 대응	로컬 LLM(HFLocal)에서도 rewrite 가능하도록 generator 재사용
엣지 케이스	로컬 모델이 빈 응답(None)을 반환하는 경우 처리
리팩터링	하드코딩된 재작성 로직을 추상 인터페이스로 통합
상태 불일치	재작성된 쿼리**(resolved_query)**가 부스팅 로직에 전달되지 않는 버그
소유권 문제	메모리 상태가 여러 모듈에 분산되어 일관성 파괴 → Retriever로 단일화
추적 가능성	디버깅 비활성화 시에도 memory_state 노출

안전 장치	잘못된 설정값(음수 토큰/타임아웃)으로 인한 무한 대기 방지

특히 상태 불일치와 소유권 문제는 단일턴 테스트에서는 절대 드러나지 않는, 멀티턴 고유의 상태 관리 버그였다. 이후 rewrite 타임아웃 시 원본 쿼리로 자동 fallback하는 안전 장치가 추가되어 멀티턴 엔진이 최종 안정화되었다.

구분	도입 전	도입 후
"그렇다면 이것은?"	원본 질문 그대로 검색 → 맥락 없는 결과	LLM rewrite로 대명사 치환 → resolved_query로 검색
실패 패턴	6.3절 실패 사례 1(Q049)	맥락 반영된 정확한 검색

핵심 교훈: 멀티턴 대화는 단일턴과 비교할 수 없는 복잡도를 가진다. 단일턴에서는 존재하지 않는 상태 관리(메모리 소유권, *resolved_query* 일관성, 타임아웃 *fallback)* 문제가 멀티턴에서만 폭발적으로 발현된다.

7.4 프롬프트 엔지니어링 체계화

7.4.1 단일 프롬프트에서 5-Tier 조합형 아키텍처로의 전환

초기 시스템은 단일 시스템 메시지를 하드코딩하여 LLM에 전달하였다. 이 방식은 프롬프트 변경 시 코드 수정이 필요하고, A/B 테스트가 불가능하며, 시나리오 A(로컬 LLM)와 시나리오 B(상용 API)에 동일한 프롬프트를 강제하는 문제가 있었다.

이를 해결하기 위해 **5-Tier** 프롬프트 아키텍처(Persona × Constraints × Format × Few-shot × CoT)를 설계하고, 각 Tier를 독립된 YAML 설정으로 분리하였다.

구분	도입 전	도입 후
프롬프트 정의	코드 내 하드코딩	YAML 1장 = 실험 1건
변경 시	코드 수정 + 재배포	YAML 수정만으로 즉시 반영
조합 실험	수동, 비체계적	**5,400**개 조합 자동 생성 후 배치 실행

5-Tier 프롬프트 아키텍처	P × C × F × S × T 조합별 독립 실험 가능	

- [추가사항] 특히, 프롬프트 엔지니어링 실험 20종의 검증 결과를 기반으로, 최종 시스템에는 **"P4_C2_F5_S3_T5"**가 최고점 조합으로 확정되었습니다. 이때 발생한 모델의 무응답 현상(Nan 반환 문제) 역시 제약조건(Constraints)을 매우 폐쇄적이고 구체적으로 가다듬어 해결함으로써 파이프라인 안정화를 입증했습니다.

7.4.2 Citation 사전 주입 시스템

LLM이 답변에서 [1], [2] 등의 출처 번호를 인용할 때, 해당 번호가 실제 검색 결과의 순서와 불일치하는 문제가 관찰되었다. 이를 해결하기 위해 **LLM**에 컨텍스트를 전달할 때 각 청크에 **[1], [2]** 번호를 사전 주입**(pre-injection)**하여, 답변의 출처 번호와 근거 카드가 정확히 1:1로 매칭되도록 보장하였다.

7.5 데이터 품질 및 인덱싱 안정화

7.5.1 Document ID 매칭 오류 해결

인덱싱 초기에 다음과 같은 데이터 품질 문제가 연쇄적으로 발생하였다:

문제	증상	해결 방법
NaN doc_id 저장	일부 청크의 문서 소속이 누락되어 검색 필터 우회	notna() 명시 체크 로직 추가
공고 번호 불일치	문서-청크 간 ID 매칭 실패로 엉뚱한 문서의 청크 반환	공고 번호 기반 document ID 통일
파일명 fallback 부재	공고 번호가 없는 문서에서 매칭 완전 실패	파일명 기반 fallback 매칭 도입
Cleaner/Validator 불안정	전처리 후 데이터 스키마 불일치	스키마 검증 및 필터 안정화

이 문제들은 개별적으로는 사소해 보이지만, 검색 필터가 엉뚱한 기관의 문서를 반환하는 치명적 오류로 이어졌다. 특히 실패 사례 1(Q049)과 같이, 질문에서 언급되지 않은 기관(고양도시관리공사)의 문서가 검색되는 현상의 근본 원인 중 하나였다.

7.6 비용 추적 및 최적화 (Cost Governance)

토큰 단위 비용 자동 추적

본 시스템은 모든 LLM 호출(생성, 재작성, 평가)에 대해 prompt_tokens, completion_tokens, total_tokens를 자동으로 기록하며, `tracking/pricing.py`의 `calc_llm_cost` 함수가 모델별 단가표를 참조하여 실시간 비용을 산출한다.

비용 발생 지점	모델	평균 비용/건
답변 생성	gpt-5-mini	$0.003~$0.010
멀티턴 재작성	gpt-5-mini	$0.001~$0.003
LLM Judge 평가	gpt-4o-mini	$0.002~$0.005
임베딩 (인덱싱)	text-embedding-3-small	$0.15 (1회성, 전체)

본 성능 분석에서 시나리오 A(로컬 모델)는 답변 생성 시 외부 상용 API 호출이 원천적으로 차단되어 있으므로(사내 L4 인프라 구동 가정), 모델 자체의 질의응답 건당 변동 비용은 **$0.00**으로 처리하였습니다. 단, 평가(Evaluation) 단계에서 파이프라인의 완성도를 자동 검증하기 위해 투입되는 Judge LLM(GPT-4o-mini 등) 채점 구동에 한해서만 약간의 부가 API 비용이 발생한다는 점이 특징입니다.

Top-K와 비용의 상관관계

6.1절의 실험 로그에서 관찰된 바와 같이, Top-K = 5에서의 건당 비용은 $0.001인 반면, Top-K = 50에서는 $0.496으로 **496**배 증가하였다. 이는 LLM에 전달되는 컨텍스트의 토큰 수가 Top-K에 비례하여 증가하기 때문이다. Top-K를 10배 늘려도 MRR 개선이 없다는 실험 결과와 결합하면, **Top-K = 5**가 비용 대비 성능의 절대적 최적점임이 재확인된다.

7.7 고도화 전후 종합 비교

프로젝트 전체의 기술적 발전을 종합하면, 시스템은 아래와 같이 변모하였다.

모듈	초기 상태	최종 상태
검색 엔진	Dense 단일	Dense + Sparse + RRF 하이브리드
재정렬	없음	Soft Boost 중심 재정렬
필터링 전략	Hard Filter	Soft Boost (MAX 0.15 상한)
멀티턴	미구현	LLM Rewrite + 슬롯 메모리 + Fallback
평가 엔진	Judge v1 (MRR=0.000)	Judge v2 (MRR=1.000)
프롬프트	단일 하드코딩	5-Tier (P×C×F×S×T) 5,400 조합
비용 추적	수동	prompt/completion 토큰 자동 로깅
사용자 **UI**	Streamlit 단일 페이지	3탭 Streamlit + Next.js 웹 UI

---

8. 종합 평가 및 제언

8.1 프로젝트 성과 종합

KPI 달성도 총괄표

KPI	목표	달성 (시나리오 **B)**	달성 **(**시나리오 **A: Gemma Top1)**	판정 **(**종합**)**

MRR	≥ 0.70	1.00	1.00	✅ 양쪽 초과 달성
Faithfulness	≥ 0.85	1.00	0.95	✅ 양쪽 초과 달성
Latency(A/B)	≤ 90/30 초	33.1초	88.2초	⚠️ 시나리오A만 달성
Cost per Query	≤ $0.05	$0.005	$0.00 (생성 비용 없음)	✅ 양쪽 달성 범위
문서 처리	101건	101건 (100%)	101건 (100%)	✅ 완료
평가셋	80문항	80문항	20문항	✅ 완료

기술적 성취 요약

달성 항목	규모
총 파싱 문서	101건 (HWP 96, PDF 4, DOCX 1)
생성 청크 수	14,035개 (인덱싱 대상 13,951개)
6종 노이즈 정제	BR태그 102,922개, PUA문자 2,831개, 중복셀 6,583개 제거
메타데이터 보정	duplicates_map 기반 발주기관명 통합, 중복 문서 인제스트 차단
12개 메타데이터 필드	기관유형, 사업도메인, 기술스택 등 자동 분류
하이브리드 검색	Dense + Sparse + RRF 융합
7단계 검색 오케스트레이션	의도 추론 → 멀티턴 → 팬아웃 → 하이브리드 → 변형 병합 → Fallback → 출력
5-Tier 프롬프트 아키텍처	P × C × F × S × T 조합별 독립 실험 가능
자동화된 실험 거버넌스	YAML 정의 → CLI 실행 → JSONL 자동 로깅 → 마크다운 리포트 생성

8.2 시나리오 A/B 모델별 실험 특성 및 한계돌파 인사이트

시나리오 특성 비교 및 최적화 성과

이번 실험을 통해 동일한 RAG 파이프라인 위에서도 LLM의 추론 체급에 따라 확연히 다른 특성을 파악하고 이를 회피하기 위한 최적화를 수행했다.

1. 상용 **API (**시나리오 **B - GPT-5-mini)**의 높은 문맥 관용성
    - Dense + Sparse + RRF와 Soft Boost가 결합된 하이브리드 검색 상위 컨텍스트를 비교적 안정적으로 소화해냈다.
    - 80개의 검증 데이터 중 MRR 1.000, Faithfulness 0.95 이상을 유지했으며, 검색 결과가 다소 넓게 주어지더라도 모델이 문맥 내 핵심 근거를 추려 답변으로 연결하는 능력이 높음을 실험으로 확인했다.
2. 로컬 **LLM (**시나리오 **A)**의 추론 한계 파악과 방어 기제 **(**프롬프트 최적화**)**
    - 4B~7B 급의 파라미터 한계로 인해, 정답 컨텍스트가 명확하게 주어져도 그 안에서 길을 잃는(Lost in the Middle) 현상과 대명사 참조 실패가 빈번하게 관찰됨 (최대 도달점 0.84수준).
    - 이를 타개하기 위해 단순 텍스트 프롬프트를 버리고 **20**종의 **5-Tier** 구조화 프롬프트 엔지니어링을 도입했다. 특히 명시적 Few-shot과 영어 기반의 CoT(단계적 추론)를 주입하고, 한국어 강제 제약을 걸었을 때 할루시네이션이 급감하는 치명적인 방어 효과를 데이터로 검증해 냈다.
    - 결과적으로 모델 파라미터를 키우지 않고도, 프롬프트와 검색 파이프라인 보정만으로 로컬 모델의 문맥 한계를 최대한 보강하는 발전을 이룩했다.

8.3 현재 한계 및 향후 고도화 로드맵

식별된 한계점

6.3절의 실패 사례 분석과 실험 로그 인과 분석을 종합하여, 현 시스템의 한계를 다음과 같이 정리한다.

한계 영역	구체적 증상	영향도	근본 원인

멀티턴 맥락 복원	"그렇다면", "이러한" 등 대명사 참조 실패 (Q049)	높음	암시적 follow-up에 대한 rewrite 강건성과 슬롯 메모리의 주제 추적 정밀도 부족
심층 검색	성능 목표 수치 등 세부 스펙이 Top-K 밖에 위치 (Q061)	높음	하이브리드 후보군 내에서도 숫자 anchor와 세부 스펙 신호의 상위 고정력이 약함
컨텍스트 내 정답 인식	정답이 청크 내에 있으나 LLM이 간과 (Q056)	중간	Lost in the Middle + 용어 미세 불일치
평가셋 품질	문서에 없는 정보를 묻는 질문 존재 (Q050, Q071)	낮음	평가셋 작성 시 정답 존재 여부 사전 검증 미흡

향후 고도화 로드맵

Phase 1: 검색 정밀도 강화 **(**단기**)**

- 숫자/단위 exact-match 및 numeric anchor 부스팅 고도화
- 직접 anchor(where_document)와 anchor_auxiliary 보조 검색의 적용 조건 정교화
- 멀티턴 rewrite의 암시적 follow-up 해석 및 슬롯 메모리 기반 주제어 추적 정밀도 개선

Phase 2: 로컬 **LLM** 품질 향상 **(**중기**)**

- RFP 도메인 특화 PEFT 파인튜닝 (LoRA/QLoRA)
- Gemma-4-E4B-it 및 Qwen2.5-7B 각각에 대해 500건 이상의 도메인 데이터로 파인튜닝
- 시나리오 A의 Faithfulness를 0.90 이상으로 끌어올릴 경우, 보안이 필요한 기관에서의 실전 배포 가능

Phase 3: 프로덕션 안정화 **(**장기**)**

- Next.js 사용자 UI와 FastAPI 백엔드의 프로덕션 배포
- 모니터링 대시보드(Latency P95, Error Rate, Cost/day) 구축
- A/B 테스트 프레임워크: 동일 질의에 대해 시나리오 A/B 답변을 병렬 생성하고 사용자 선호도 피드백 수집

8.4 결론

BidMate RAG 프로젝트는 공공입찰 RFP 101건을 대상으로, 데이터 무결성 확보 → 구조 보존형 전처리 → 하이브리드 검색 오케스트레이션 → Soft Boost 기반 재정렬 → 5-Tier 프롬프트 기반 생성이라는 엔드투엔드 파이프라인을 구축하여, 시나리오 B에서 MRR 1.000, 건당 비용 $0.001 (Top-K=5 기준)이라는 정량적 성과를 달성하였다.

사례 분석에서 확인된 바와 같이, 성공 사례의 100%가 메타데이터 필터의 정확한 작동과 환각 방지 Constraints의 준수라는 두 축 위에서 달성되었다. 반면, 실패 사례의 대부분은 검색 단계(Retrieval)의 한계 또는 멀티턴 맥락 복원 실패에서 기인하였으며, LLM의 환각으로 인한 실패는 단 한 건도 관찰되지 않았다.

이는 RAG 시스템의 성능 향상 전략이 "더 큰 LLM"이 아니라 "더 정밀한 검색"에 집중되어야 함을 시사한다. BidMate RAG 시스템은 이 원칙에 충실하여, 7단계 검색 오케스트레이션과 Soft Boost 중심의 재정렬 파이프라인이라는 높은 수준의 검색 정밀도 아키텍처를 구현하였다.

시나리오 A(로컬 LLM)와 시나리오 B(상용 API)의 이원화 전략은, 보안 요건과 비용 구조가 극단에 있는 조직 모두에게 적합한 솔루션을 제공한다. 특히 BEP 분석(일일 2,619건)을 통해, 각 조직의 상황에 맞는 합리적 엔진 선택이 가능함을 정량적으로 입증하였다.

BidMate RAG는 “질문에 대해 문서 근거를 명시하며 정확하게 답변하는” 지식 근거주의(Evidence-grounded) 체계로서, 공공입찰 컨설팅의 디지털 전환을 위한 견고한 기술적 토대를 확립하였다.

8.4종합 결론 및 시사점

BidMate RAG 프로젝트는 방대한 RFP 문서들을 대상으로 단순히 검색 엔진을 단 것이 아니라, 데이터 클리닝 → 의미망 기반 청킹 → 하이브리드**(Dense+BM25+RRF)** 결합 → **Soft Boost** 최적화 → **5-Tier** 프롬프트 통제라는 과감하고 지속적인 기술적 개선 여정을 밟아왔다.

가장 뼈아팠던 초기의 0.000 MRR 사태를 딛고, 단순 문자열 매칭의 한계를 깨달아 **LLM** 기반 역동적 평가 엔진**(Judge v2)**으로 전면 교체함으로써 파이프라인의 진정한 성능을 눈으로 확인할 수 있었다. 또한 한때 기대를 모았던 Cross-Encoder 가 오히려 A/B 공정성을 훼손하고 리소스 병목을 유발한다는 사실을 데이터로 직시하고 과감하게 배제하는 엔지니어링 결단을 통해, 불필요한 과부하 없이 MRR 1.0 가까이 솟구치는 거대한 개선을 이루어냈다.

실험이 거듭될수록 검색 기술이 아무리 정교해져도, 결국 압도적인 컨텍스트를 소화해 내는 시나리오 **B(**상용 **API)**와 정보 속에서 길을 잃고 마는 시나리오 **A(**로컬 모델**)** 사이의 근본적인 추론 장벽은 존재했다**.** 하지만 우리는 이 벽 앞에서 포기하지 않고 20번의 세밀한 프롬프트 깎기(5원칙 확립)와 영문 CoT 강제화를 통해 로컬 모델의 잠재력을 최대한 쥐어짜 내어 그 간극을 최대한 메우는 데 성공했다.

이제 파이프라인 레벨에서의 구조적 최적화는 한계점에 도달했다. 현재 파악해 낸 시나리오 A의 태생적 한계(4B 파라미터의 추론 제약성)를 바탕으로 남은 넥스트 스텝의 방향성은 자명하다. 단순히 프롬프트를 수정하는 수준을 넘어, 도메인 특화 데이터를 활용한 **PEFT(LoRA)** 파인튜닝 실험을 시도하는 것이다. 물론 소형 모델의 파인튜닝이 반드시 극적인 추론력 상승으로 이어질지는 미지수이나, 프롬프트 최적화의 한계점을 극복하고 폐쇄망 환경에서도 온전하게 자생 가능한 RAG 생태계를 구축할 수 있을지에 대한 마지막 검증(Hypothesis Testing) 단계로서 필수적인 도약이 될 것이다.